IA Générative Multi Modale
Introduction à l'IA Générative, Modèles Génératifs

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité



IA Discriminative — *Apprendre à distinguer*

L'IA discriminative apprend la frontière entre les classes (ou la relation entrée → sortie) à partir de données étiquetées. Elle prédit une étiquette ou une valeur pour une nouvelle donnée, mais ne génère pas de nouvelles données.


Chat


Chien


Oiseau


Voiture

...

Données étiquetées
(exemples d'apprentissage)

Données d'entrée (x)

Modèle discriminatif
(ex. CNN, ResNet, SVM, ...)



Apprend P(y | x)

Prédiction
(Classe ou valeur)

Chat	0.92
Chien	0.05
Oiseau	0.02
Voiture	0.01

...

Tâches typiques :
Classification, détection, régression, reconnaissance, ...

VS

IA Générative — *Apprendre à créer*

L'IA générative apprend la distribution des données P(x) (ou P(x | condition)) et est capable de générer de nouvelles données réalistes et cohérentes, telles que des images, du texte, de l'audio, de la vidéo ou du code.

Prompt / Condition (exemple)

- « Un chat portant un nœud papillon, assis dans un salon élégant »
- « A scenic mountain landscape at sunset »
- « Écris un court poème sur l'espoir »

...

Modèle génératif
(ex. GAN, VAE, Diffusion, Transformers, ...)



Apprend P(x) ou P(x | condition)

Nouvelle donnée générée


Image générée


Image générée

Dans chaque aube naît une chance
Un rêve qui se lève et avance
L'espoir éclaire même la nuit
Et fait de demain un cœur qui vit

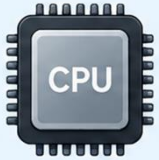
Texte généré

...

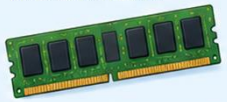
Tâches typiques :
Génération d'images, de texte, d'audio, de vidéo, de code, de données synthétiques, ...

CPU (Hôte)

Traitement général, préparation des données



RAM (mémoire principale)
Stocke les données, le code Python, les tensors sur CPU, etc.



- Exécute le code Python (PyTorch)
- Prépare et prétraite les données
- Gère l'entraînement (boucle, optimisation, ...)
- Peut effectuer les calculs sur CPU (si pas de GPU)

.to("cuda") — Transfert de données — .to("cpu")

GPU (Device)

Calculs massivement parallèles pour les tenseurs



VRAM (mémoire du GPU)
Stocke les tensors, les paramètres du modèle, les activations, ...



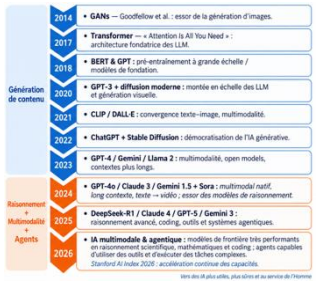
- Exécute les opérations sur les tensors (très parallèles)
- Idéal pour l'entraînement des grands modèles
- Mémoire dédiée : la VRAM (plus rapide que la RAM)
- Nécessite CUDA (NVIDIA) pour être accessible depuis PyTorch

CUDA

Couche logicielle de NVIDIA

PyTorch (torch.cuda)
↓
CUDA Runtime (cuBLAS, cuDNN, ...)
↓
Pilote NVIDIA (Driver)

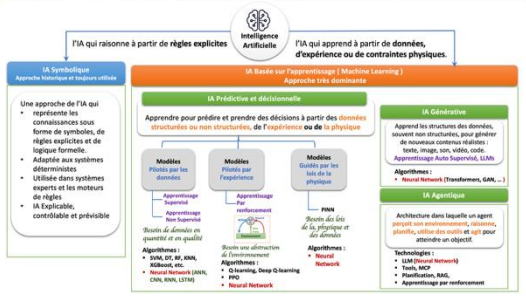
NVIDIA



QWEN-3

Gemma 4





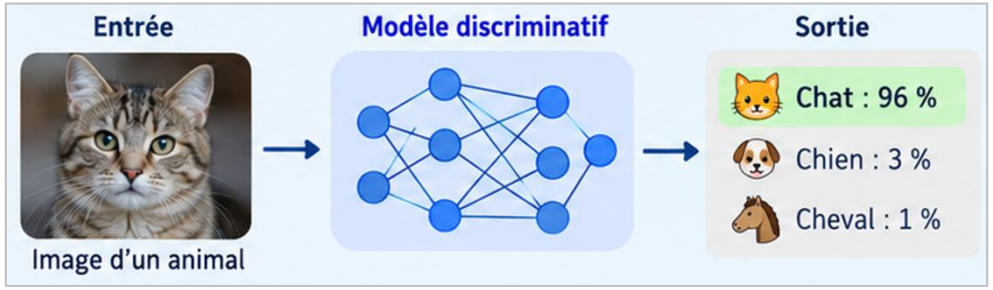
IA Discriminative

Vs

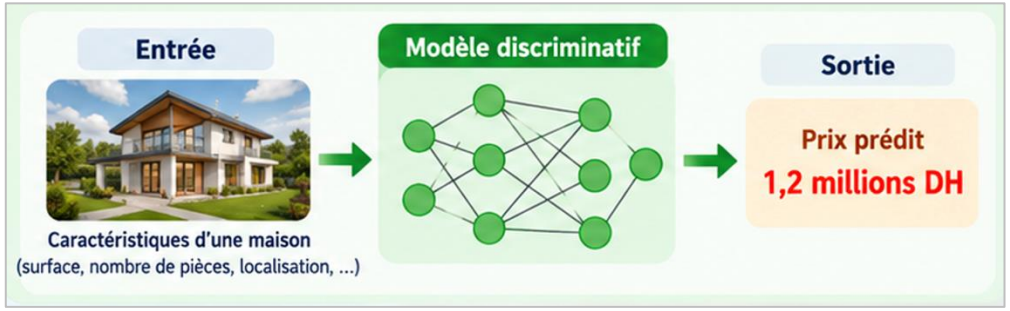
IA Générative

Cherche apprendre à partir de données, à établir une relation entre des entrées et des sorties afin de **prédire** **une catégorie (classification)** ou **une valeur numérique (régression)** pour de nouvelles données.

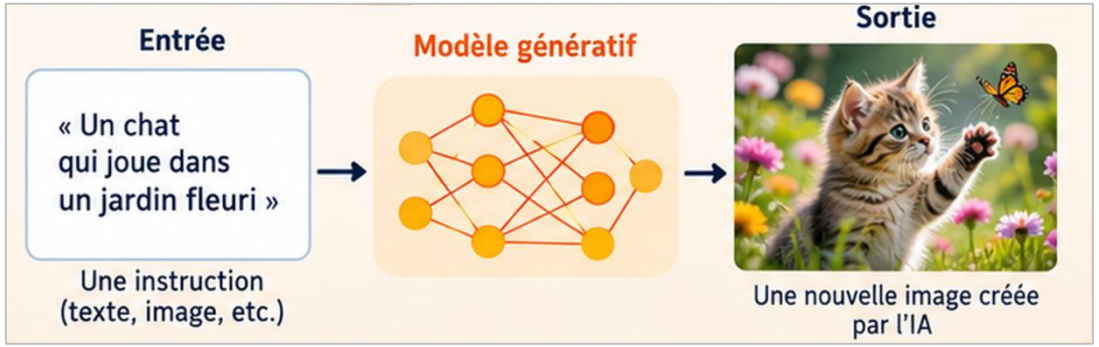
Classification :



Régression :



Cherche à apprendre à partir de données d'entraînement, les structures, les caractéristiques et les distributions statistiques de ces données afin de **générer** de nouveaux contenus présentant des propriétés similaires à celles des données apprises.





Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)

Auto encodeur variationnel (VAE)

Un auto encodeur variationnel est un modèle génératif qui apprend **une représentation latente continue et probabiliste des données**, ce qui permet de **reconstruire**, de **générer** et d'**interpoler** de nouvelles données.



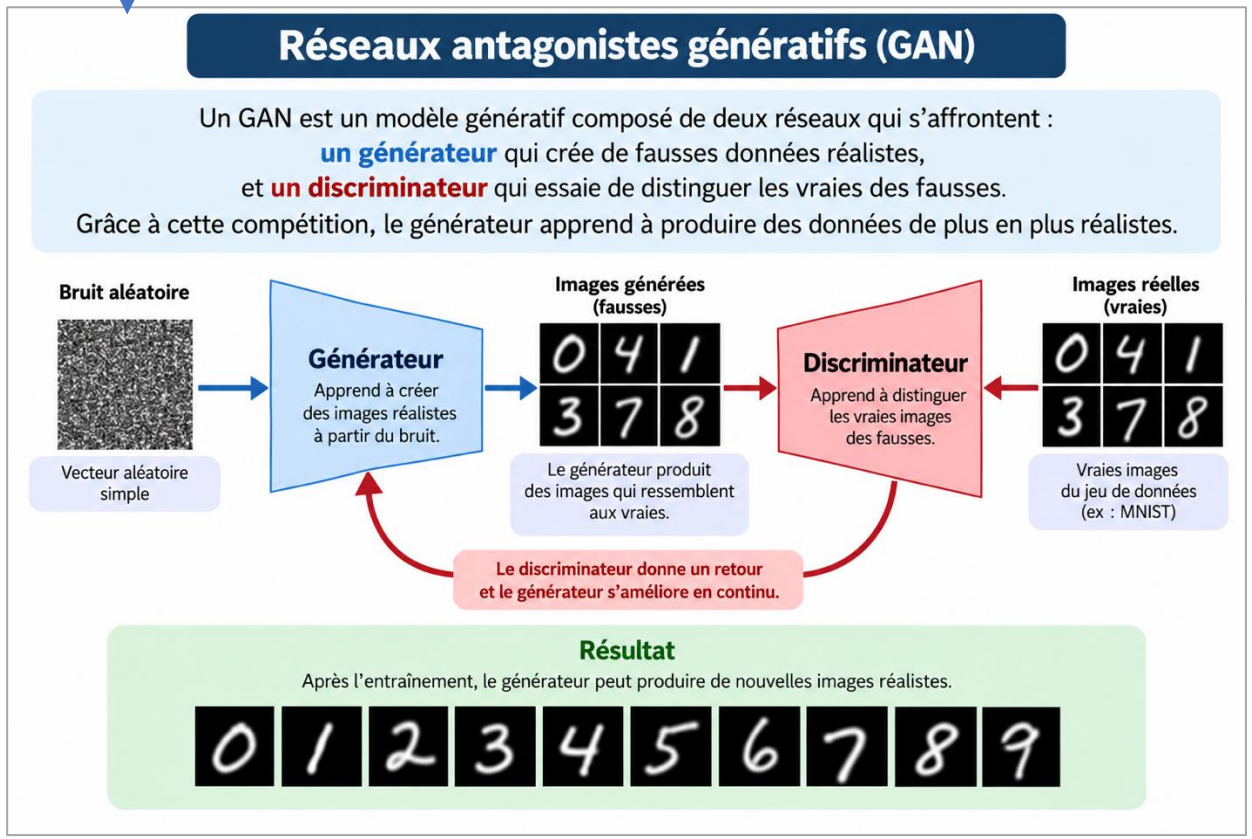
Génération de nouvelles données

En échantillonnant de nouveaux points dans l'espace latent, on peut générer de nouvelles images.



Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)



https://thispersondoesnotexist.com/

Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

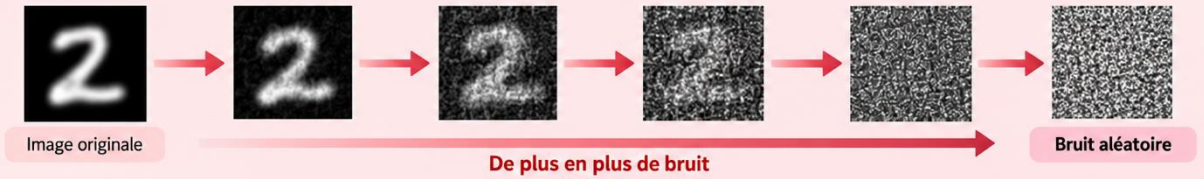
- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)

Modèles de diffusion

Un modèle de diffusion est un modèle génératif qui apprend à **supprimer progressivement le bruit** pour **reconstruire** des données réalistes.
Après l'apprentissage, il peut générer de nouvelles données en partant d'un bruit aléatoire.

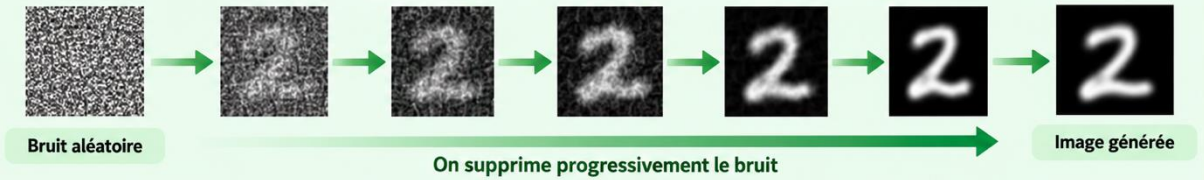
1. Processus direct : ajout de bruit

On ajoute progressivement du bruit à une image jusqu'à obtenir du bruit aléatoire.



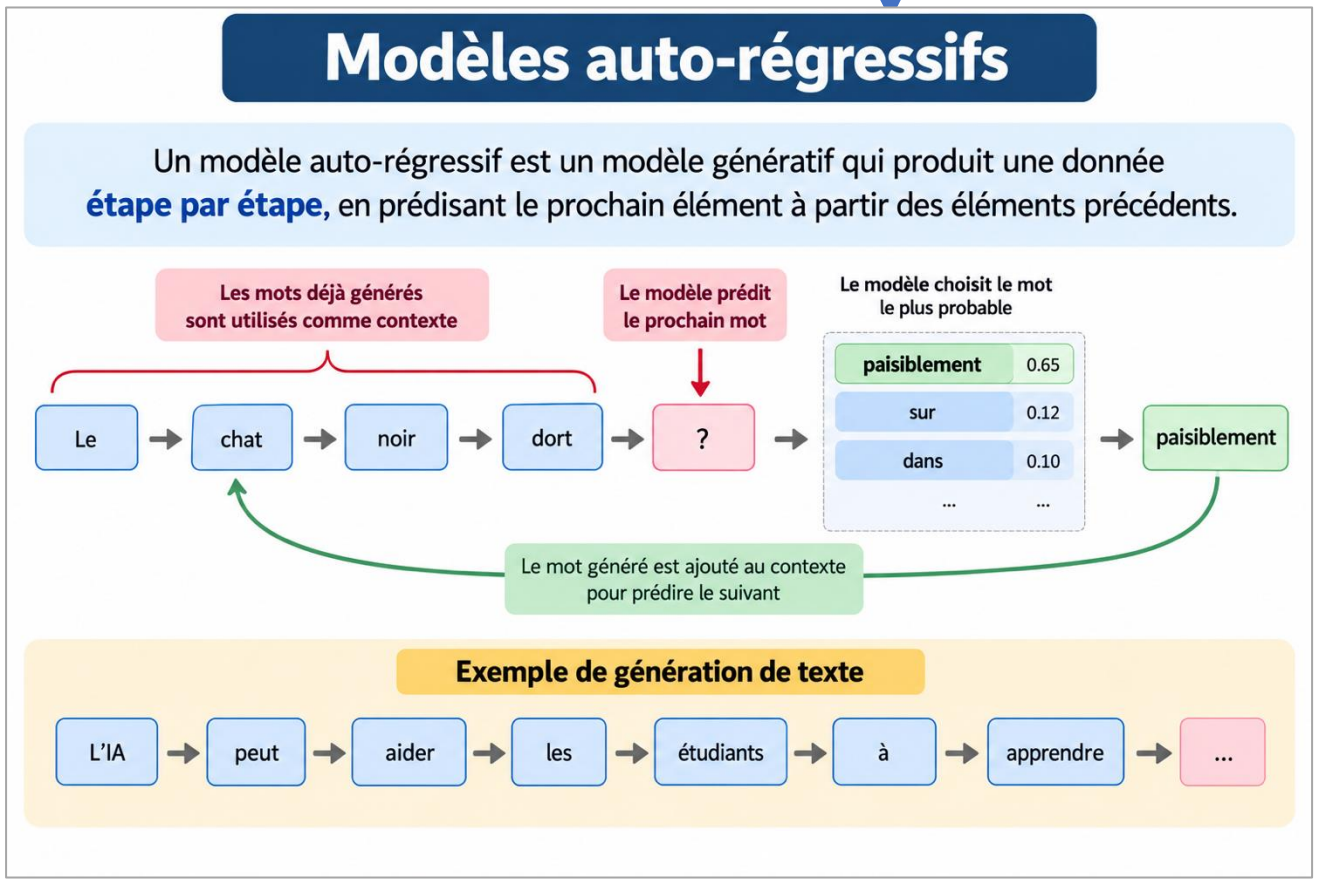
2. Processus inverse : suppression du bruit

Le modèle apprend à supprimer le bruit pas à pas pour retrouver une image réaliste.



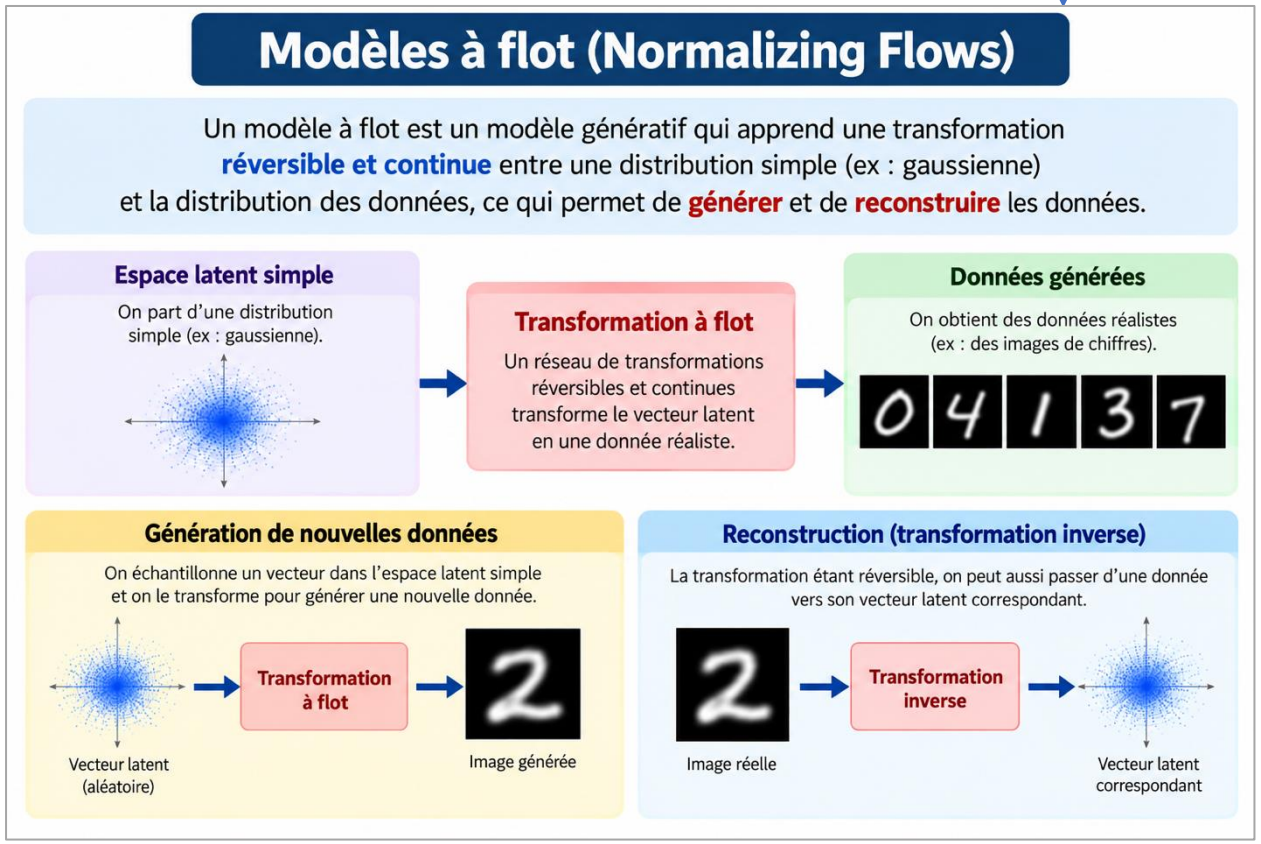
Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)



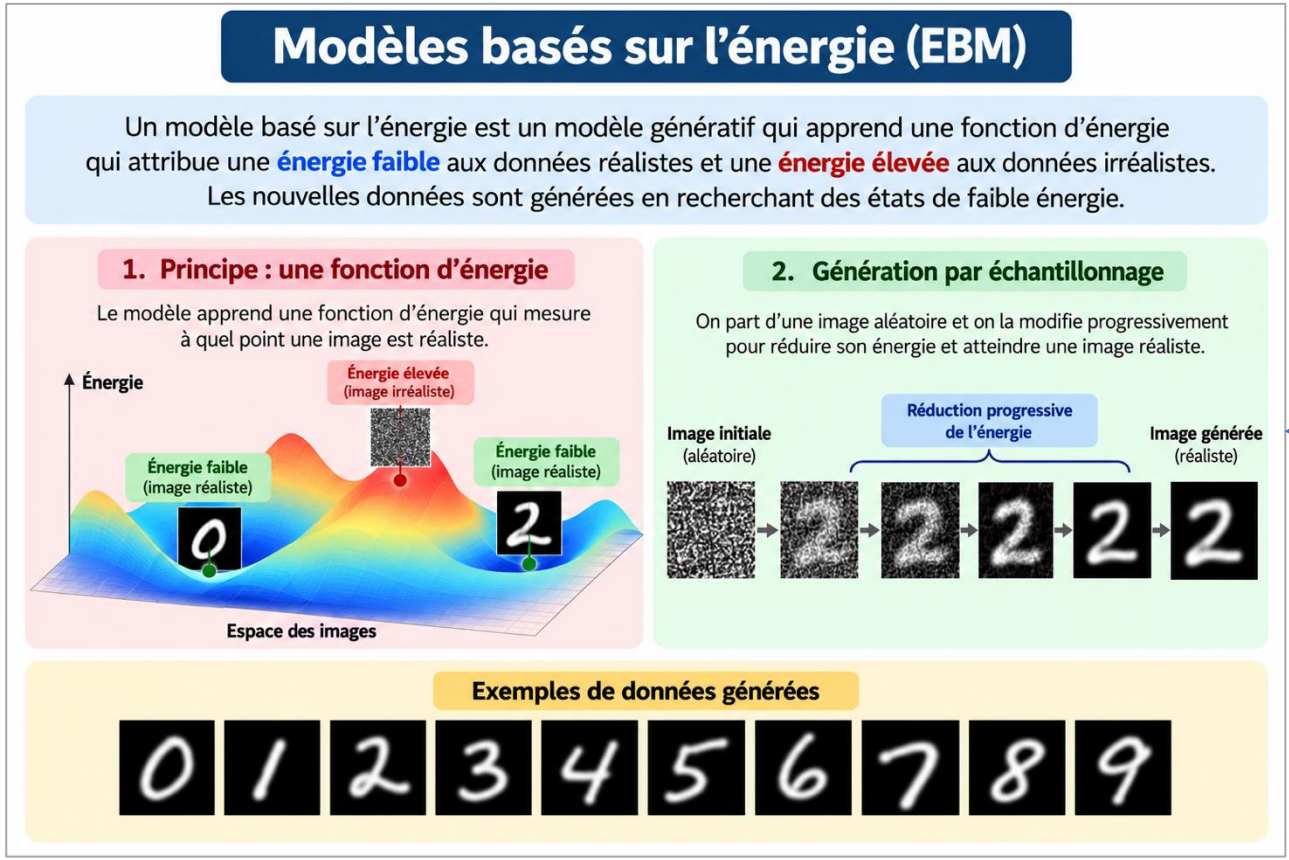
Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)



Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)



Modèles Génératifs
Apprendre une distribution des données pour générer de nouveaux exemples

- Variational Autoencoders (VAE)
- Generative Adverserial Networks (GAN)
- Diffusion Models (DF)
- Autoregressive Models (ARM)
- Flow-based Models (FBM)
- Energy Based Models (EBM)

Classification selon l'objectif

Non Conditionnelle
Générer à partir d'un bruit ou d'une variable latente



Conditionnelle
Générer à partir d'une condition (label, prompt)



Multimodale
Générer une modalité à partir d'une autre (Text -> image)



Par transformation
Modifier un contenu existant (images -> image)



Multi Modale unifiée
Modifier un Traiter et générer plusieurs modalités (texte, image, audio, vidéo)



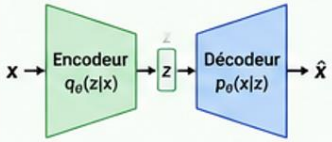
Modèles génératifs

Apprendre une distribution des données pour générer de nouveaux exemples

Autoencodeurs variationnels (VAE)

Espace latent probabiliste

Apprennent une représentation latente et générert en échantillonnant cet espace.





Applications
Représentation latente, images, données synthétiques

Réseaux antagonistes génératifs (GAN)

Générateur contre discriminateur

Un générateur crée des données synthétiques et un discriminateur tente de les distinguer des réelles.



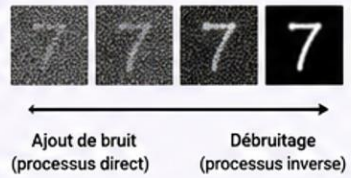


Applications
Images réalistes, visages, super-résolution

Modèles de diffusion

Bruit puis débruitage

Apprennent à inverser un processus de corruption par le bruit pour générer des données.

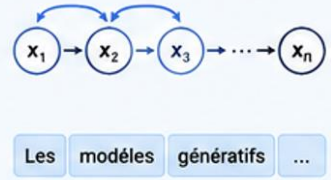


Applications
Images, audio, vidéo, haute qualité et diversité

Modèles autorégressifs

Prédiction séquentielle

Générerent chaque élément conditionnellement aux éléments précédents.

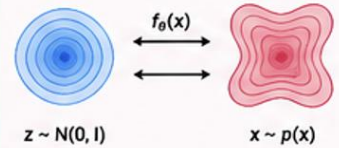


Applications
Texte, code, musique, images tokenisées

Modèles de flot

Transformations inversibles

Transforment une distribution simple en une distribution complexe via des fonctions inversibles.

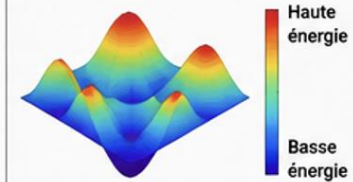


Applications
Estimation de densité, génération de données

Modèles basés sur l'énergie (EBM)

Énergie des configurations

Apprennent une function d'énergie qui attribue une faible energie aux configurations plausibles.



$$p(x) = \frac{1}{Z} e^{-E_\theta(x)}$$

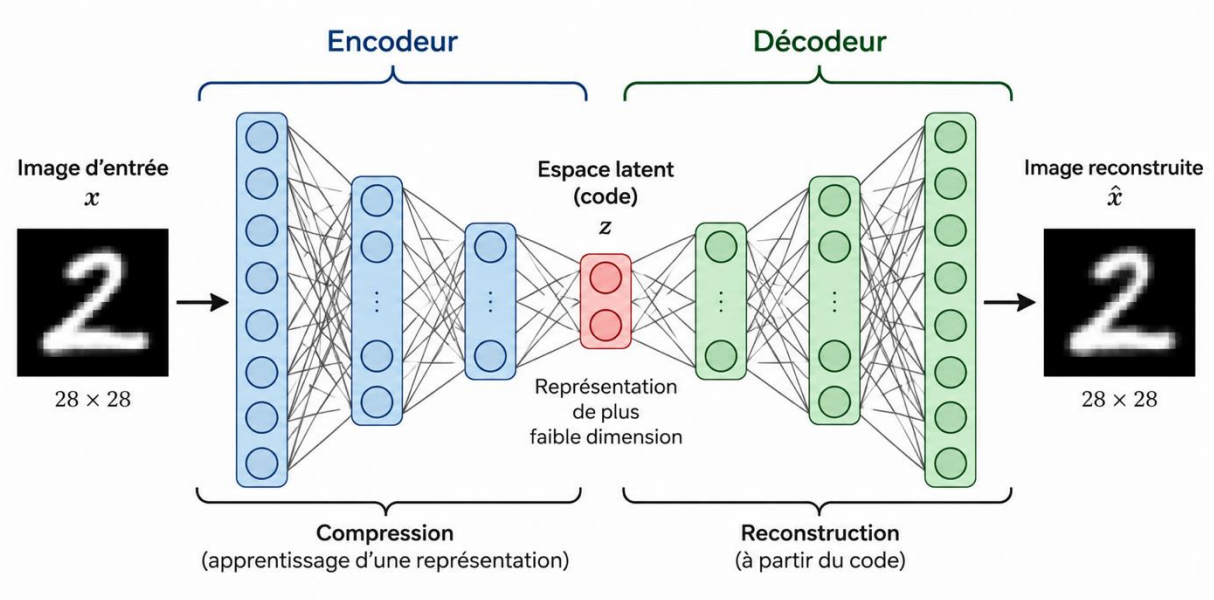
Applications
Modélisation flexible, détection d'anomalies

Tableau comparatif

Famille	Mécanisme fondamental	Atout principal	Limite principale	Exemples
VAE	Espace latent probabiliste	Représentation structurée	Images parfois floues	VAE, β-VAE
GAN	Apprentissage adversarial	Images très réalistes	Instabilité, mode collapse	GAN, DCGAN, StyleGAN
Diffusion	Débruitage itératif	Qualité et diversité	Coût de génération	DDPM, Latent Diffusion, Stable Diffusion
Autorégressif	Prédiction conditionnelle	Texte et séquences	Génération séquentielle	GPT, PixelCNN, WaveNet
Flows	Transformations inversibles	Densité exacte tractable	Contraintes architecturales	Real NVP, Glow
EBM	Fonction d'énergie	Modélisation flexible	Échantillonnage coûteux	RBM, EBM

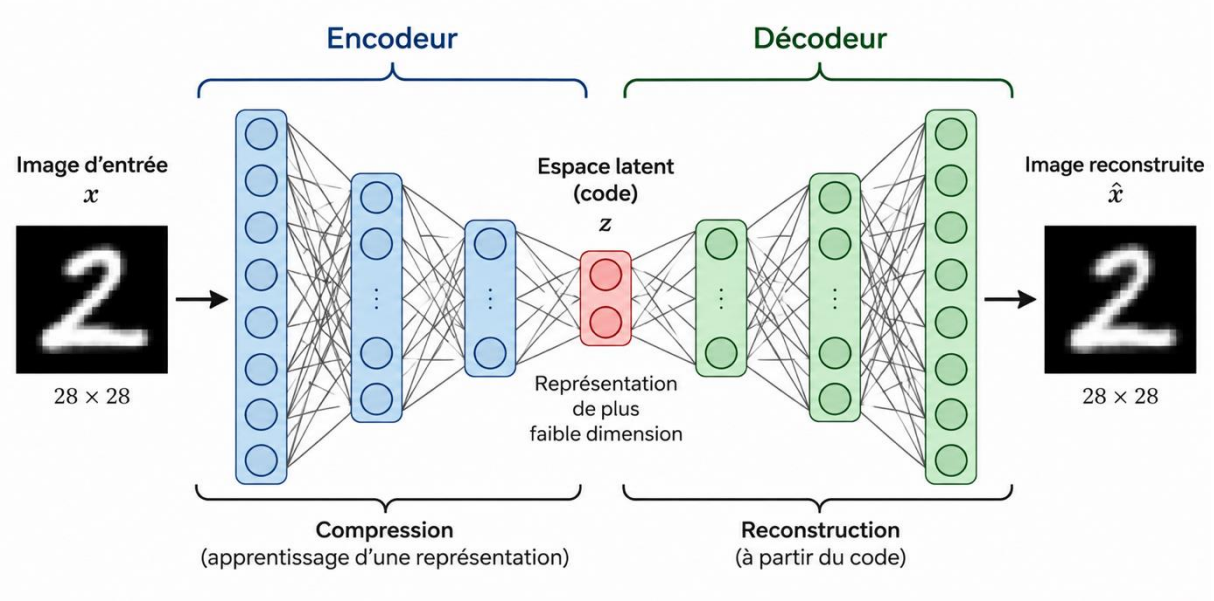
Auto-Encodeur

- La reconstruction est un objectif d'apprentissage. Elle oblige le modèle, lorsqu'on lui impose des contraintes adaptées, à conserver les informations utiles dans une représentation interne.
- L'objectif n'est donc pas uniquement de copier une entrée, mais d'apprendre une représentation exploitable des données.
- **Question :** Si vous deviez décrire une image avec seulement quelques nombres, quelles informations faudrait-il conserver ?



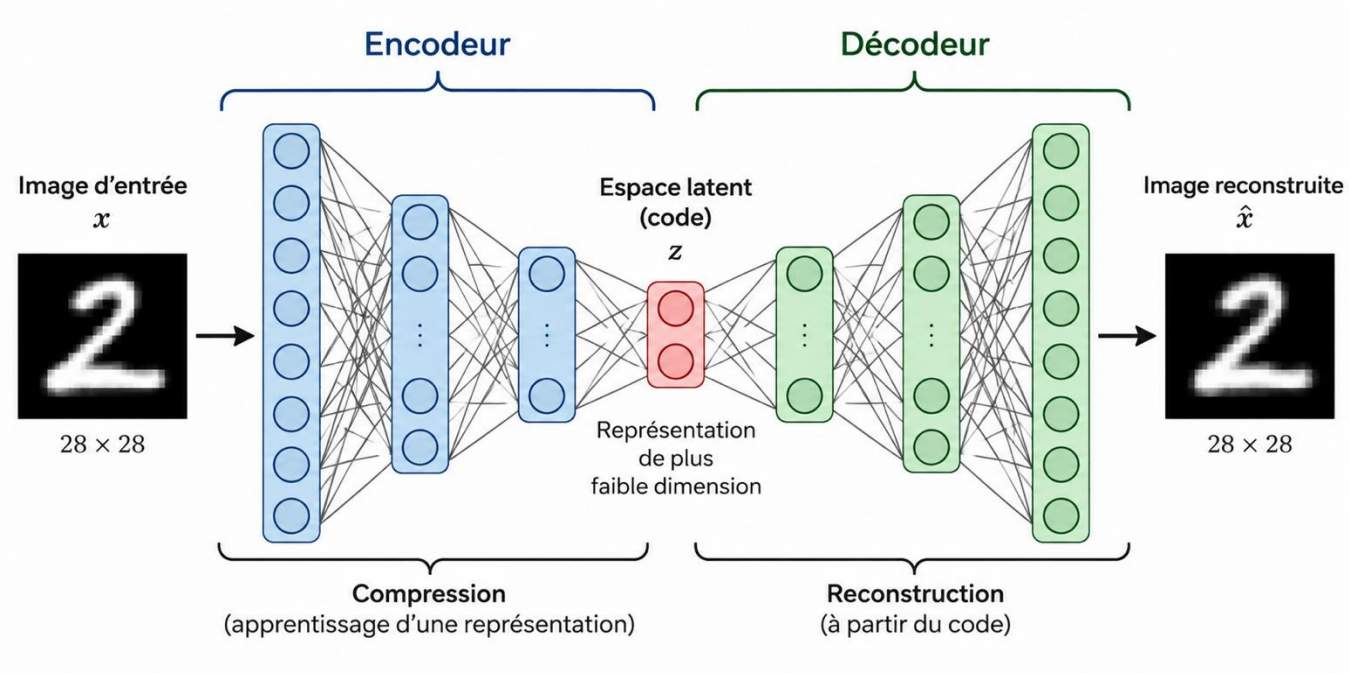
Auto-Encodeur

- Si vous deviez décrire une image avec seulement quelques nombres, quelles informations faudrait-il conserver ?
  - conserver les informations essentielles qui permettent de reconnaître l'image et d'en reconstruire une approximation : la forme générale des objets, leurs contours, leur position, leur orientation et, si elles sont importantes, leurs couleurs. Les détails fins ou le bruit pourraient être laissés de côté. »
  - Exemple d'un chiffre manuscrit : quelques nombres pourraient décrire la présence d'une boucle, l'inclinaison, la courbure et l'épaisseur des traits, plutôt que mémoriser chaque pixel.
  - C'est le rôle de la **représentation latente** : encoder les informations utiles à la reconstruction dans un petit nombre de valeurs. Le réseau apprend lui-même ce codage ; chaque valeur ne correspond pas nécessairement à une caractéristique identifiable.



Étapes d’entraînement

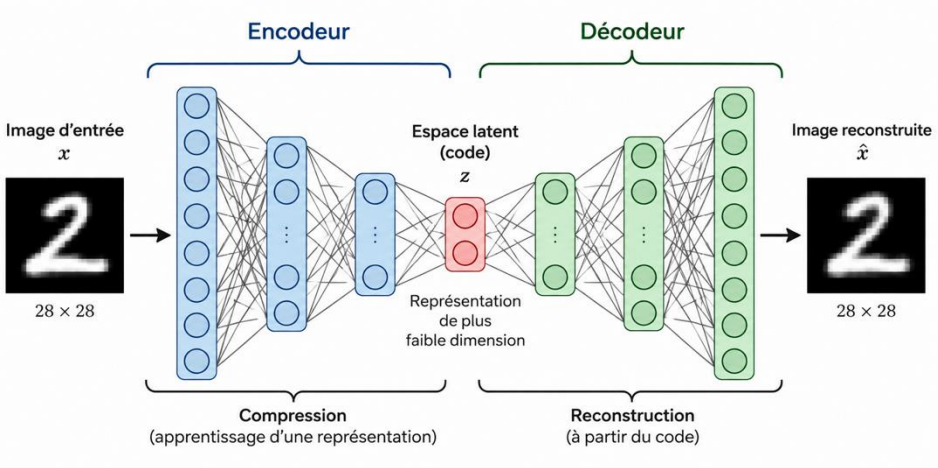
1. Sélectionner un lot de données.
2. Calculer les représentations latentes.
3. Reconstruire les entrées.
4. Calculer la perte de reconstruction.
5. Rétro-propager les gradients.
6. Mettre à jour les paramètres.

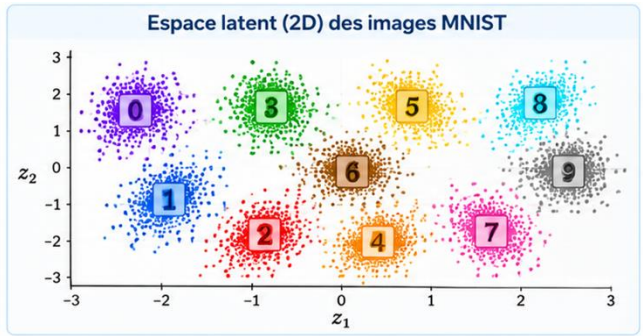


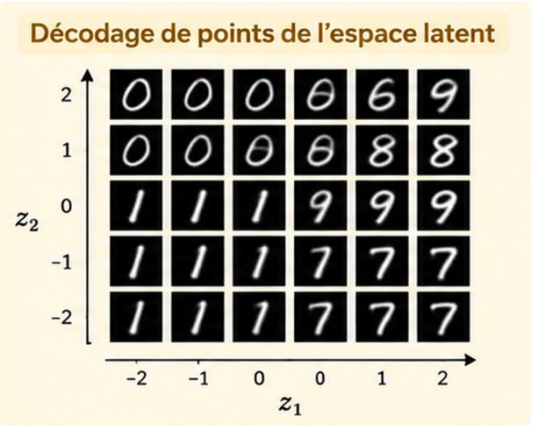
[Animation Auto Encodeur vs VAE]

Comprendre la représentation latente

- **Que contient le code latent ?**
  - Le vecteur (z) représente les informations que le modèle utilise pour reconstruire l'entrée.
  - Pour des chiffres manuscrits, il peut notamment refléter :
    - l'inclinaison ;
    - l'épaisseur des traits ;
    - la courbure ;
    - la forme générale du chiffre.
  - Cependant, **rien ne garantit qu'une coordonnée latente corresponde à une caractéristique interprétable**. L'information peut être répartie sur plusieurs dimensions.







Influence de la dimension latente

Dimension latente	Effet possible
Trop faible	Perte d'informations et reconstruction dégradée
Intermédiaire	Compromis entre compression et fidélité
Très élevée	Reconstruction facilitée, mais représentation parfois peu structurée

Une grande capacité peut permettre au réseau d'apprendre une fonction proche de l'identité, voire de mémoriser les données d'entraînement.

Implémentation d'un auto-encodeur

Chargement des données

- https://ossci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz
- https://ossci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz
- https://ossci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz
- https://ossci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader

from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

print("PyTorch :", torch.__version__)

device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
print("Device :", device)
```

```
transform = transforms.ToTensor()

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

print("Train :", len(train_dataset))
print("Test  :", len(test_dataset))
```

```
data
└── mnist
    └── raw
        ├── t10k-images-idx3-ubyte
        ├── t10k-images-idx3-ubyte.gz
        ├── t10k-labels-idx1-ubyte
        ├── t10k-labels-idx1-ubyte.gz
        ├── train-images-idx3-ubyte
        ├── train-images-idx3-ubyte.gz
        ├── train-labels-idx1-ubyte
        └── train-labels-idx1-ubyte.gz
```

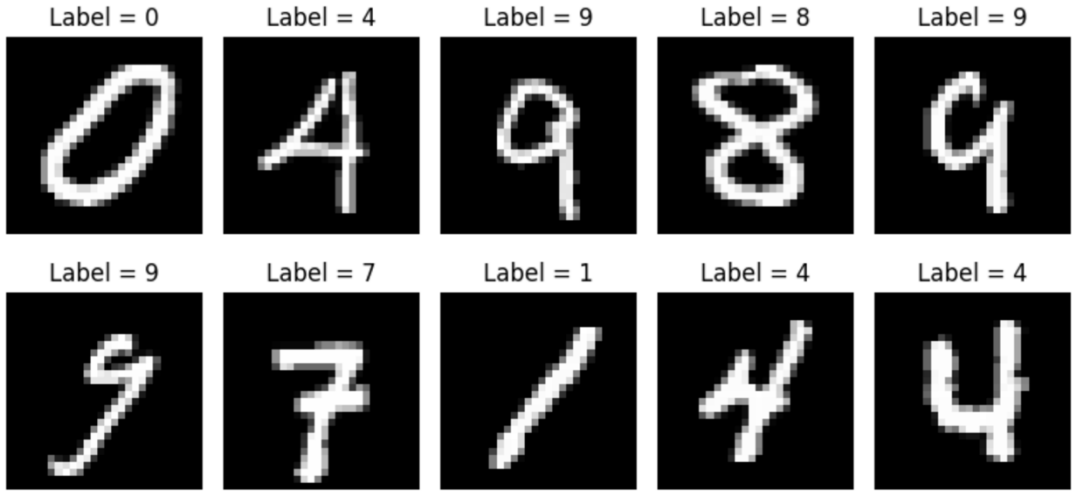
```
Train : 60000
Test  : 10000
```

Implémentation d'un auto-encodeur

Affichage des données

```
images, labels = next(iter(train_loader))

fig, axes = plt.subplots(2, 5, figsize=(8, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(images[i].squeeze(), cmap="gray")
    ax.set_title(f"Label = {labels[i].item()}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```



Modèle discriminatif

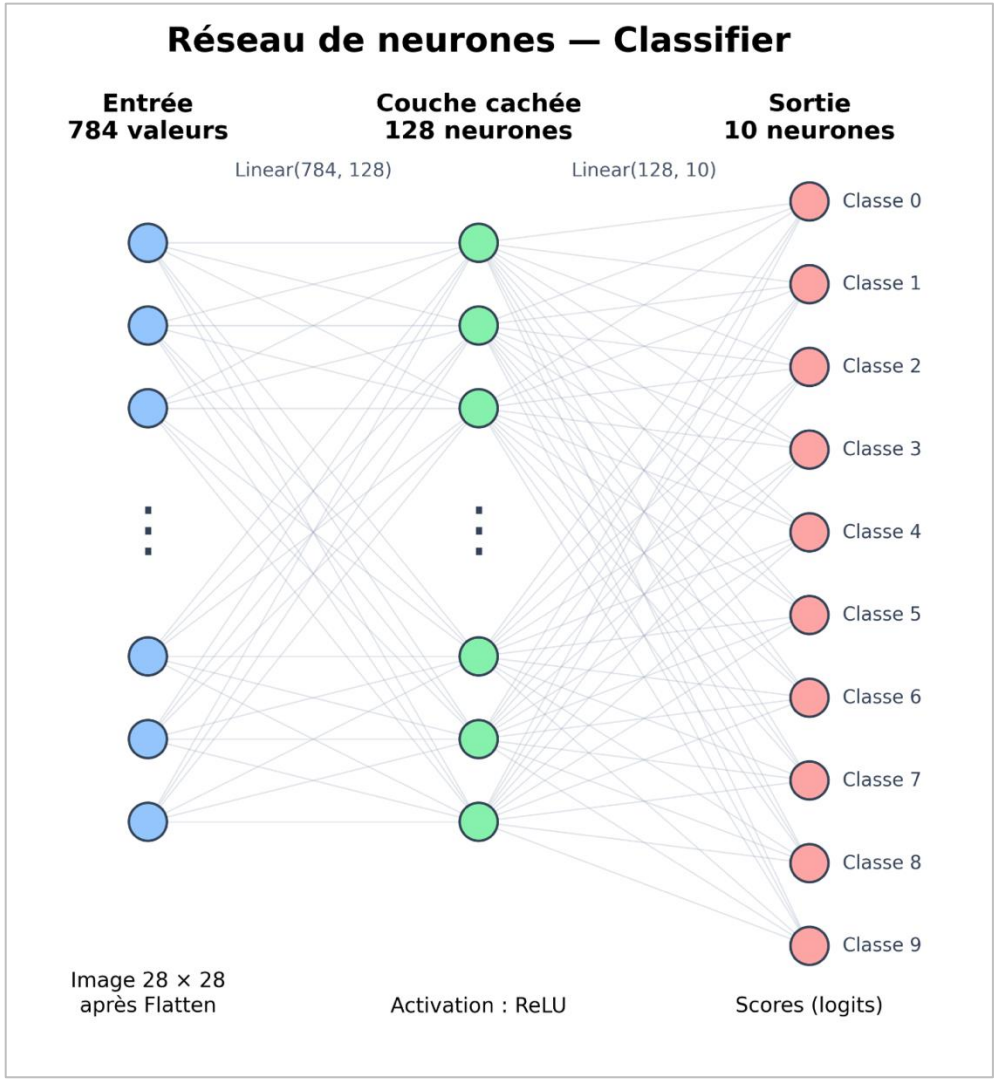
Modèle discriminatif

```
class Classifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        return self.net(x)

classifier = Classifier().to(device)
classifier
```

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(classifier.parameters(), lr=1e-3)
```





Modèle discriminatif

Entrainement

```
def train_classifier(model, loader, epochs=3):
    model.train()

    for epoch in range(epochs):
        total_loss = 0

        for x, y in loader:
            x, y = x.to(device), y.to(device)

            logits = model(x)
            loss = criterion(logits, y)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        print(
            f"Epoch {epoch+1}/{epochs} - "
            f"Loss = {total_loss/len(loader):.4f}"
        )
train_classifier(classifier, train_loader, epochs=3)
```

```
Epoch 1/3 - Loss = 0.4073
Epoch 2/3 - Loss = 0.1934
Epoch 3/3 - Loss = 0.1408
```

Evaluation

```
def evaluate_classifier(model, loader):
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)

            logits = model(x)
            pred = logits.argmax(dim=1)

            correct += (pred == y).sum().item()
            total += y.size(0)

    return correct / total

accuracy = evaluate_classifier(classifier, test_loader)
print(f"Accuracy = {100*accuracy:.2f}%")
```

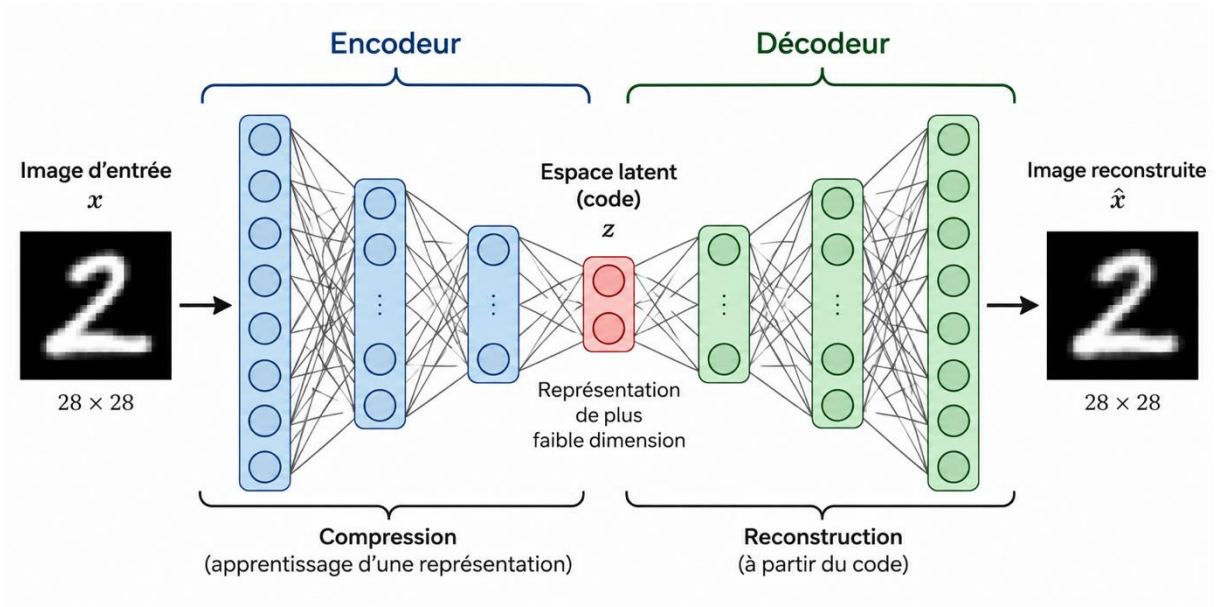
```
Accuracy = 96.41%
```

Modèle Génératif : Auto-encodeur

```
class Autoencoder(nn.Module):
    def __init__(self, latent_dim=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 32),
            nn.ReLU(),
            nn.Linear(32, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 128),
            nn.ReLU(),
            nn.Linear(128, 784),
            nn.Sigmoid()
        )
    def encode(self, x):
        return self.encoder(x)

    def decode(self, z):
        x_hat = self.decoder(z)
        return x_hat.view(-1, 1, 28, 28)

    def forward(self, x):
        z = self.encode(x)
        x_hat = self.decode(z)
        return x_hat
ae = Autoencoder(latent_dim=2).to(device)
ae
```




```
Autoencoder(
  (encoder): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=32, bias=True)
    (4): ReLU()
    (5): Linear(in_features=32, out_features=2, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=2, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=784, bias=True)
    (5): Sigmoid()
  )
)
```

Modèle Génératif : Auto-encodeur

```
ae_criterion = nn.MSELoss()
ae_optimizer = torch.optim.Adam(ae.parameters(), lr=1e-3)
```

```
def train_autoencoder(model, loader, epochs=5):
    model.train()

    for epoch in range(epochs):
        total_loss = 0

        for x, _ in loader:
            x = x.to(device)

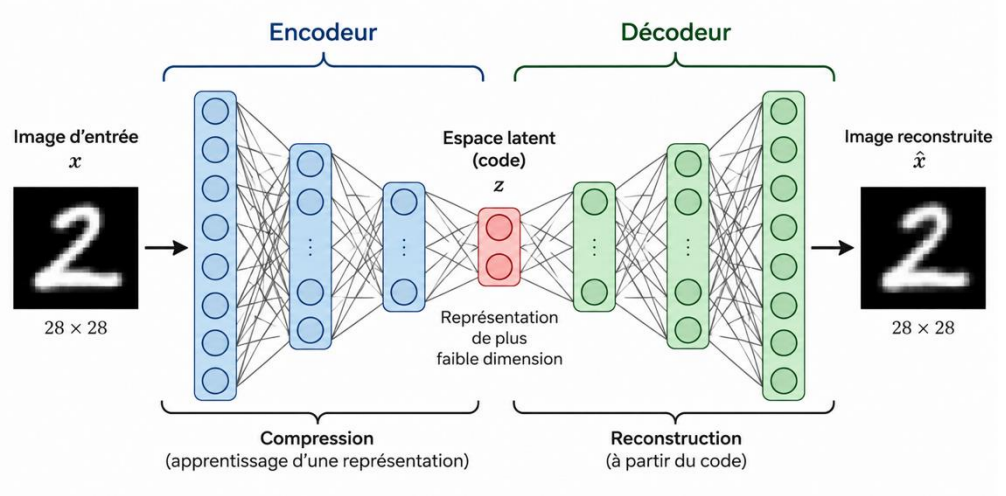
            x_hat = model(x)
            loss = ae_criterion(x_hat, x)

            ae_optimizer.zero_grad()
            loss.backward()
            ae_optimizer.step()

            total_loss += loss.item()

        print(
            f"Epoch {epoch+1}/{epochs} - "
            f"Loss = {total_loss/len(loader):.4f}"
        )

train_autoencoder(ae, train_loader, epochs=5)
```




```
Epoch 1/5 - Loss = 0.0657
Epoch 2/5 - Loss = 0.0511
Epoch 3/5 - Loss = 0.0477
Epoch 4/5 - Loss = 0.0457
Epoch 5/5 - Loss = 0.0446
```

Modèle Génératif : Auto-encodeur

```
ae.eval()

x, _ = next(iter(test_loader))
x = x[:10].to(device)

with torch.no_grad():
    x_hat = ae(x)

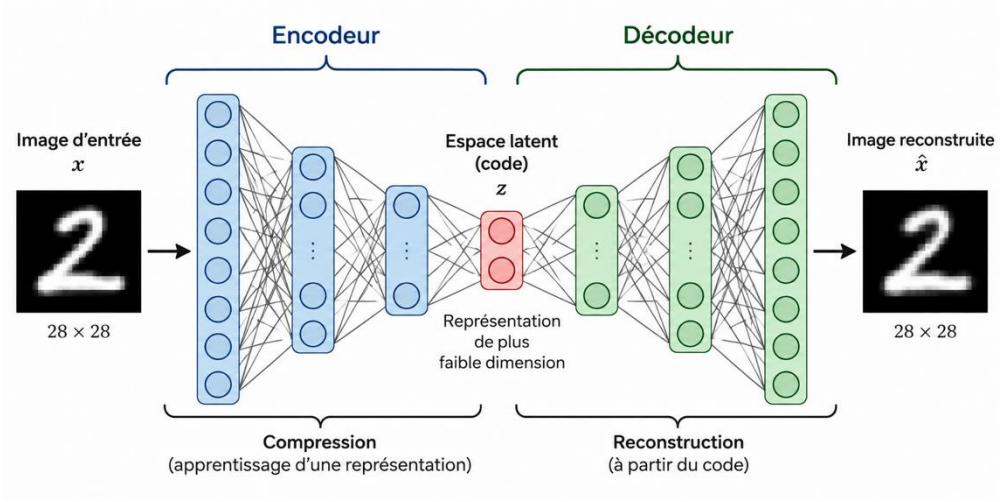
fig, axes = plt.subplots(2, 10, figsize=(15, 3))

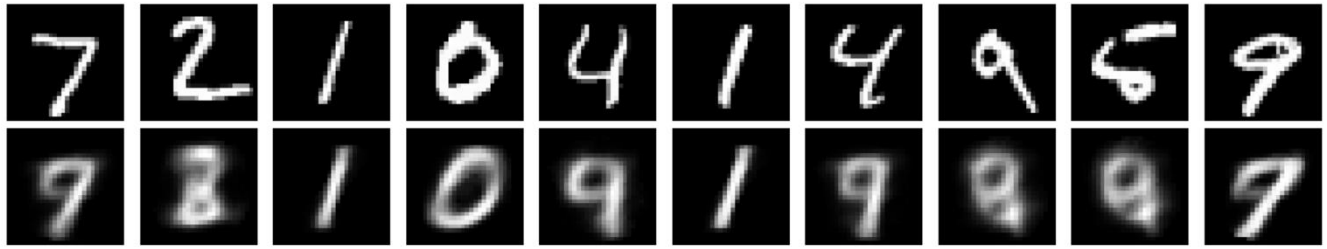
for i in range(10):
    axes[0, i].imshow(x[i].cpu().squeeze(), cmap="gray")
    axes[0, i].axis("off")

    axes[1, i].imshow(x_hat[i].cpu().squeeze(), cmap="gray")
    axes[1, i].axis("off")

axes[0, 0].set_ylabel("Original")
axes[1, 0].set_ylabel("Reconstruit")

plt.tight_layout()
plt.show()
```





Modèle Génératif : Auto-encodeur

```
ae.eval()

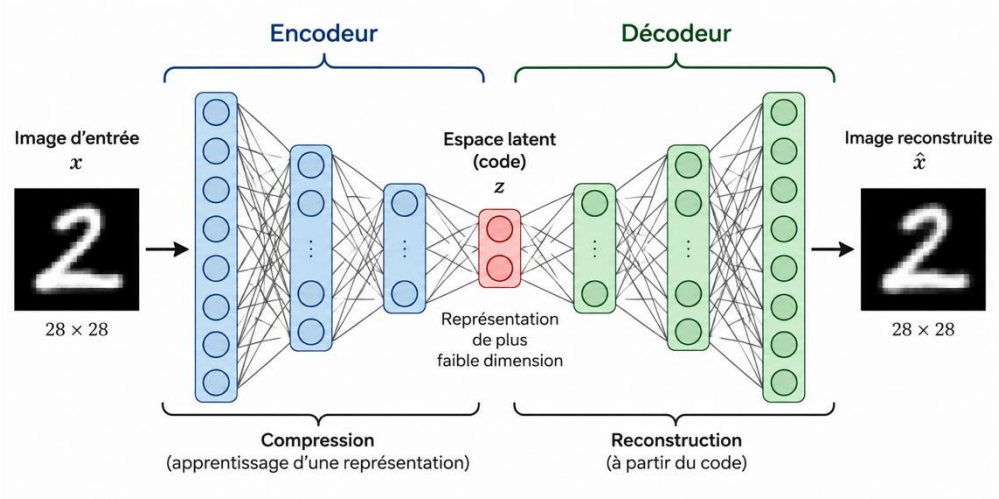
zs = []
ys = []

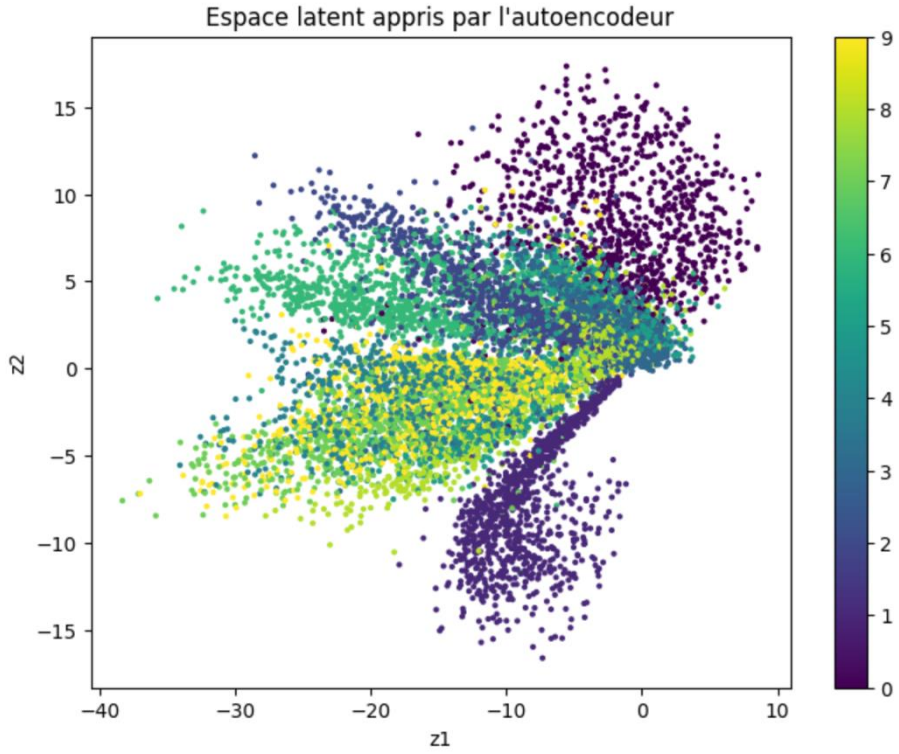
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(device)
        z = ae.encode(x)

        zs.append(z.cpu())
        ys.append(y)

Z = torch.cat(zs).numpy()
Y = torch.cat(ys).numpy()

plt.figure(figsize=(8, 6))
scatter = plt.scatter(Z[:, 0], Z[:, 1], c=Y, s=4)
plt.xlabel("z1")
plt.ylabel("z2")
plt.title("Espace latent appris par l'autoencodeur")
plt.colorbar(scatter)
plt.show()
```





Variantes principales des auto-encodeurs

Variante	Principe	Intérêt
Sous-complet	Limiter la dimension latente	Réduction de dimension
Débruitant	Reconstruire une donnée propre à partir d'une entrée corrompue	Robustesse au bruit
Parcimonieux	Pénaliser certaines activations latentes	Représentations peu actives
Convolutionnel	Utiliser des convolutions	Exploiter la structure spatiale des images
Variationnel — VAE	Apprendre une distribution latente régularisée	Génération par échantillonnage

Applications

- **Réduction de dimension** et visualisation.
- **Extraction de caractéristiques** pour une tâche ultérieure.
- **Débruitage** d'images ou de signaux.
- **Détection d'anomalies** par erreur de reconstruction.
- **Compression**, en ajoutant les mécanismes de quantification et de codage nécessaires.

Limites :

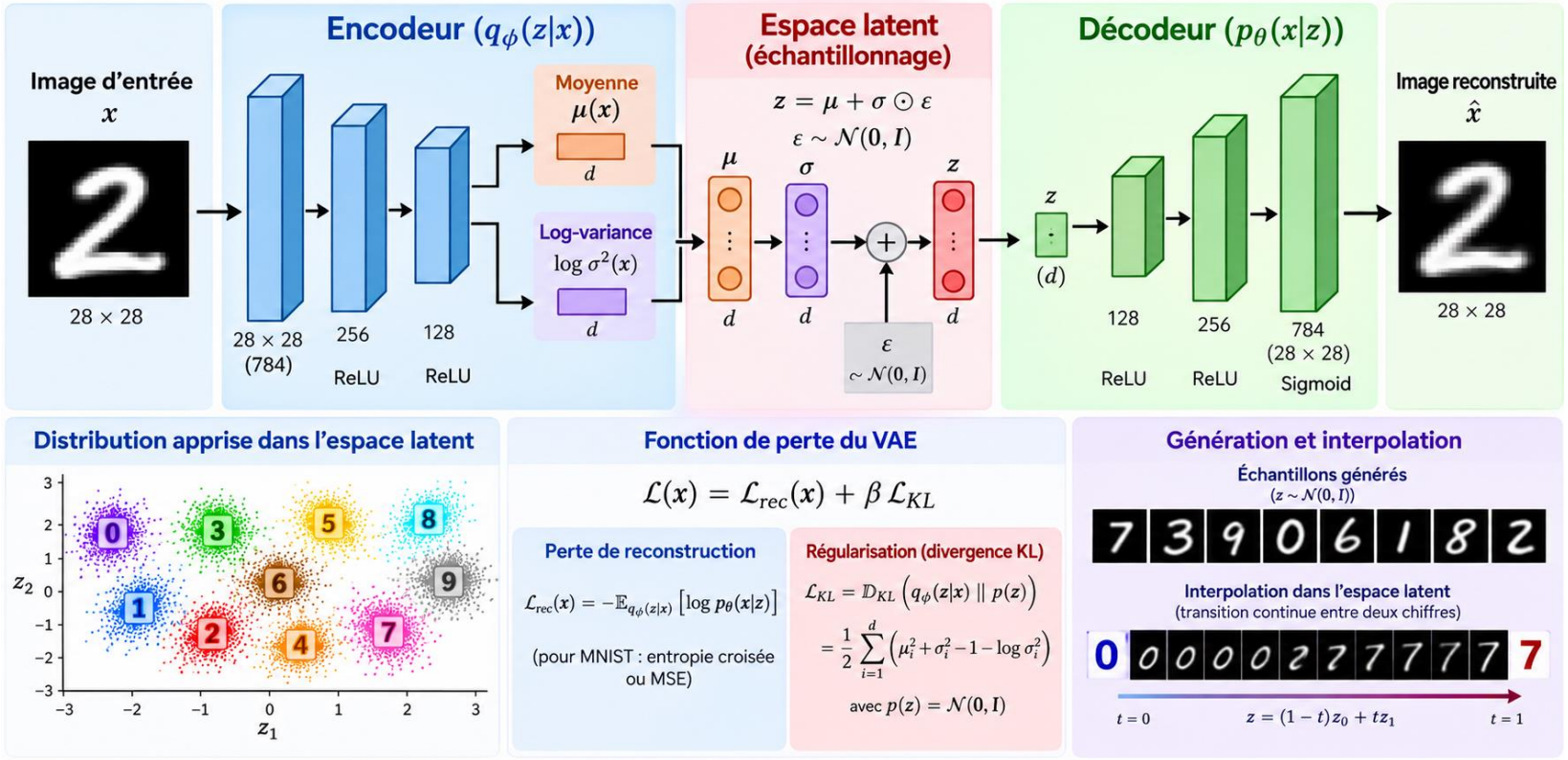
- Une anomalie peut parfois être bien reconstruite.
- Une faible MSE ne garantit pas une bonne qualité perceptuelle.
- L'espace latent n'est pas nécessairement interprétable.
- Les performances dépendent du prétraitement et de la capacité du réseau.
- La reconstruction sur les données d'entraînement ne suffit pas à évaluer le modèle.

Modèle Génératif : Variational Auto Encoder (VAE)

Le **Variational Autoencoder**, ou **auto-encodeur variationnel**, introduit une approche probabiliste :

- l'encodeur produit une **distribution latente** pour chaque entrée ;
- on échantillonne un code dans cette distribution ;
- le décodeur utilise ce code pour modéliser les données ;
- une régularisation rapproche les distributions latentes d'une distribution de référence.

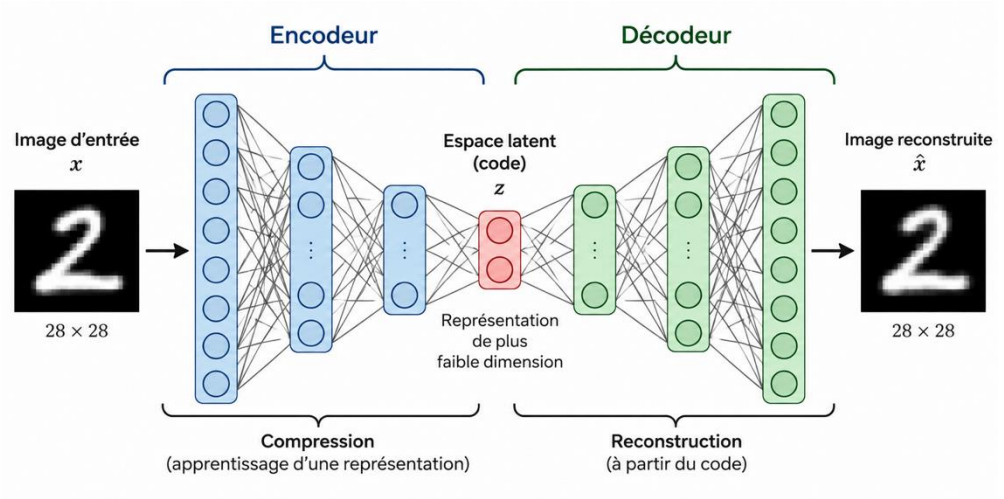
Cette organisation permet de générer de nouvelles données en échantillonnant dans la distribution de référence.



Modèle Génératif : Variational Auto Encoder (VAE)

Limite d'un auto-encodeur classique

- Un auto-encodeur contient deux réseaux :
  - **l'encodeur**, qui transforme une image en un petit vecteur de nombres ;
  - **le décodeur**, qui utilise ce vecteur pour reconstruire l'image.
  - Image d'un « **2** » (**784 pixels**)→ Encodeur → Code latent (**2 nombres**) → Décodeur → Reconstruction du **« 2 » (784 pixels)**
  - Une image MNIST contient 28 × 28 = 784 pixels. Pour illustrer, imaginons que l'encodeur la résume avec seulement deux nombres :
    - Code latent = [1.2, -0.7]
    - Ce code constitue une **représentation compacte** de l'image.
    - Pendant l'entraînement, on cherche à rendre la reconstruction aussi proche que possible de l'image originale.

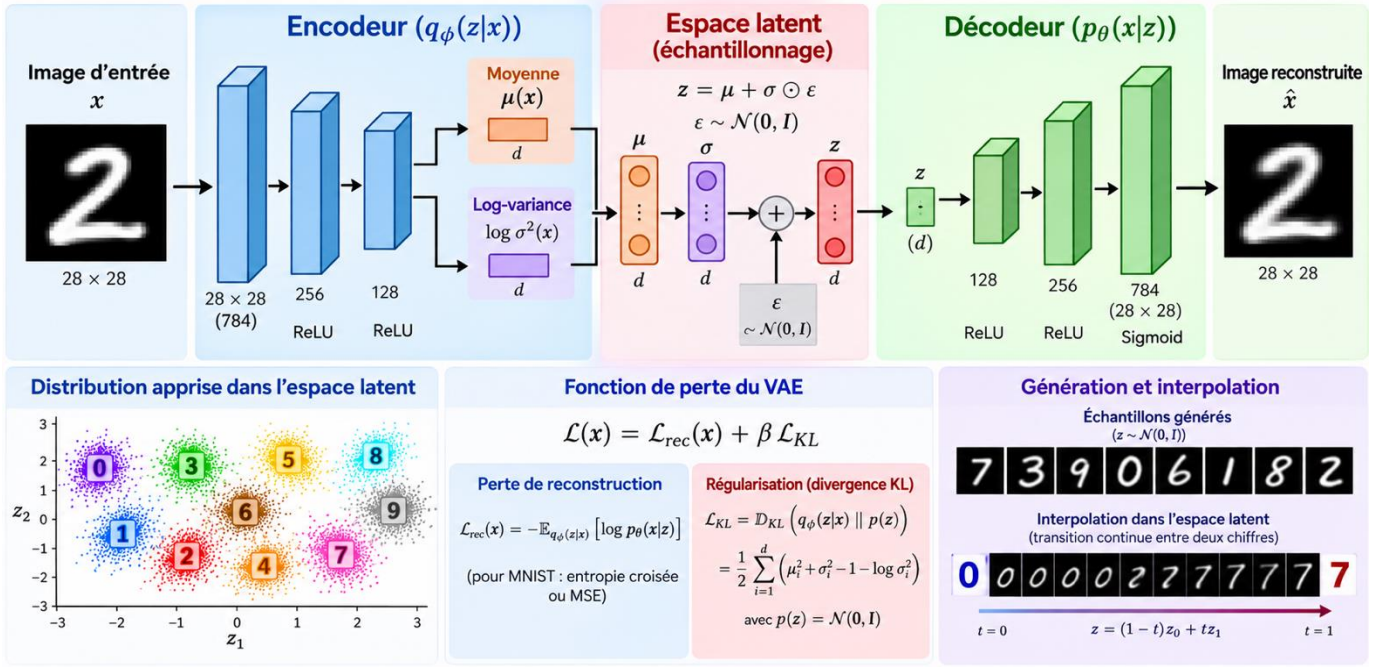


Pourquoi cela ne suffit-il pas pour générer ?

- Le décodeur apprend à reconstruire les codes produits par l'encodeur. Mais il n'apprend pas nécessairement à donner un résultat cohérent pour **n'importe quel code choisi au hasard**.
- Si on lui donne [8.5, -12.0], il peut produire une image sans sens.
- **Il manque une organisation de l'espace latent permettant de savoir où tirer de nouveaux codes.**
- C'est précisément ce que le **VAE** cherche à apporter.

Modèle Génératif : Variational Auto Encoder (VAE)

- **Dans un VAE, l'encodeur produit une distribution**
- Au lieu de dire : « Cette image correspond exactement au point [1.2, -0.7]. »
- l'encodeur dit : « Cette image correspond à une région autour de [1.2, -0.7], avec une certaine dispersion. »
- Pour décrire cette région, on utilise généralement une **distribution gaussienne**, aussi appelée **distribution normale**.
- L'encodeur produit deux vecteurs :
  - **la moyenne** (**μ**), qui indique le centre ;
  - **l'écart-type** (σ), qui indique la dispersion dans chaque direction.
  - Par exemple :
    - μ = [1.2, -0.7]
    - σ = [0.2, 0.3]

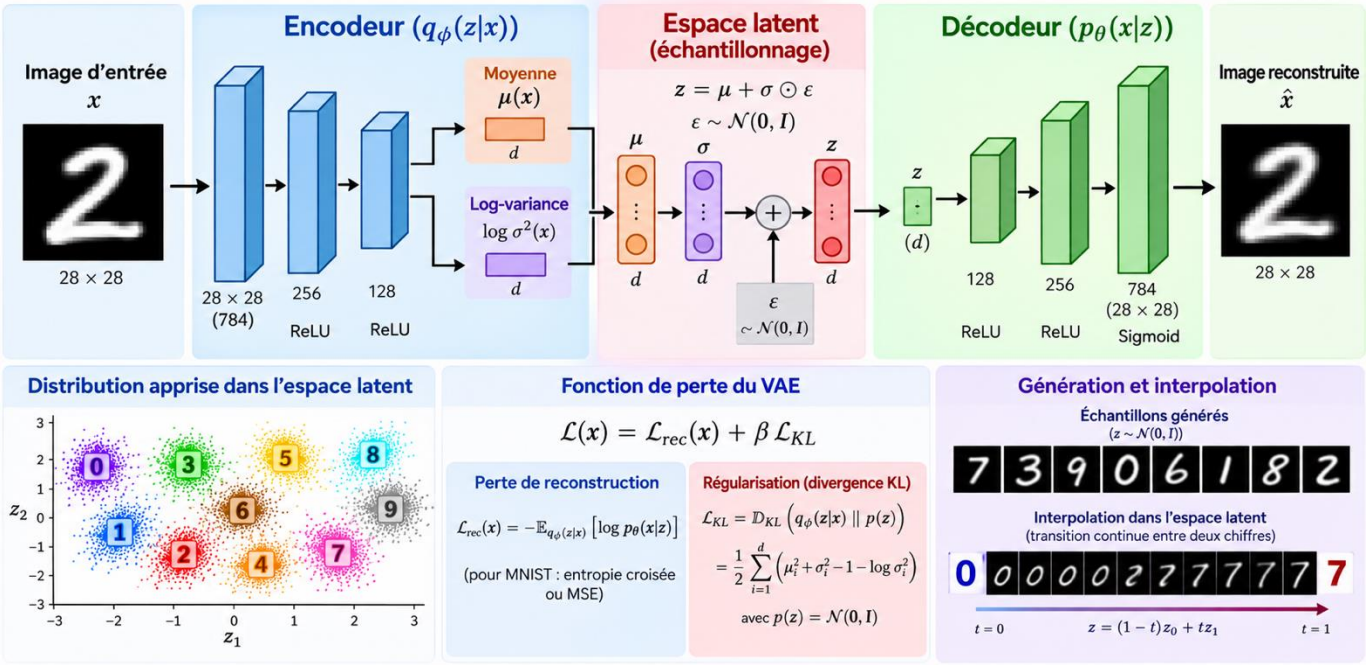


- Les codes tirés seront généralement proches de [1.2, -0.7], mais pas toujours identiques.
- **Image mentale :** un auto-encodeur classique place chaque image sur un point ; un VAE lui associe un petit nuage de points.
- Chaque image possède son propre nuage, calculé par l'encodeur.

Modèle Génératif : Variational Auto Encoder (VAE)

- **On échantillonne un code dans ce nuage**
  - Une fois la distribution obtenue, on tire un code latent (z).
  - Pour la même image, différents tirages pourraient donner :
    - Premier tirage : z = [1.10, -0.65]
    - Deuxième tirage : z = [1.35, -0.90]
    - Troisième tirage : z = [1.22, -0.41]
  - On transmet ensuite ce code au décodeur :
  - Image → Encodeur → μ et σ → Tirage de z → Décodeur → Reconstruction
  - L'objectif est que ces différents codes permettent de reconstruire une image proche de l'entrée.
  - Cela encourage le décodeur à fonctionner dans une **région** de l'espace latent, plutôt qu'en un seul point précis.
- **Comment réalise-t-on ce tirage ?**
  - On utilise la formule :

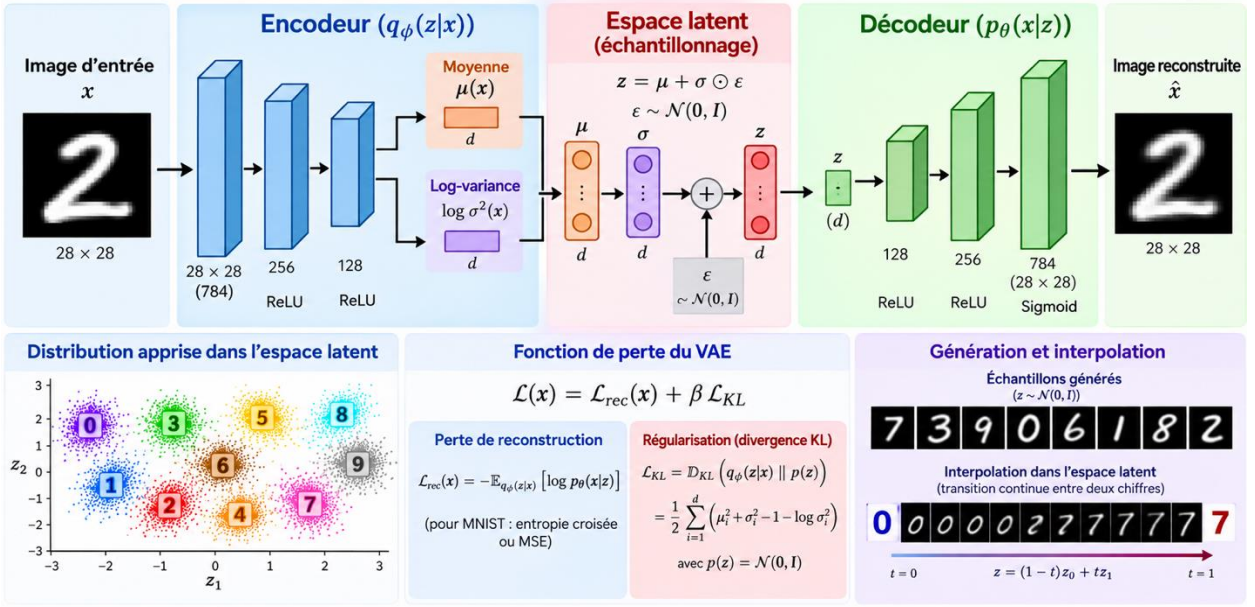
$$[\, z = \mu + \sigma \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)]$$



- Autrement dit :
  - on tire un bruit gaussien standard (***ε***) ;
  - on ajuste son amplitude avec (σ) ;
  - on le déplace autour du centre (μ).
  - Le symbole (⊙) signifie « multiplication élément par élément ».
  - Cette méthode s'appelle **l'astuce de reparamétrisation**. Elle permet de conserver le tirage aléatoire tout en calculant les gradients nécessaires pour entraîner l'encodeur.

Modèle Génératif : Variational Auto Encoder (VAE)

- **Le décodeur apprend à reconstruire les données**
  - Le décodeur reçoit (z) et produit les paramètres d'une distribution sur les images.
  - Pour MNIST, dans une version courante, il produit une valeur entre 0 et 1 pour chaque pixel : on peut l'interpréter comme une probabilité d'activation du pixel.
  - En pratique, on affiche souvent directement ces valeurs comme une image en niveaux de gris.
  - Code latent z → Décodeur → Valeurs des pixels → Image reconstruite
  - On mesure ensuite la différence entre l'original et la reconstruction grâce à une **perte de reconstruction**.
  - Cette perte demande au modèle : « Conserve suffisamment d'information pour retrouver l'image de départ. »
  - Mais cette contrainte seule ne suffit toujours pas à organiser l'espace latent.



Modèle Génératif : Variational Auto Encoder (VAE)

La régularisation organise les nuages

- On choisit une distribution de référence simple, généralement :

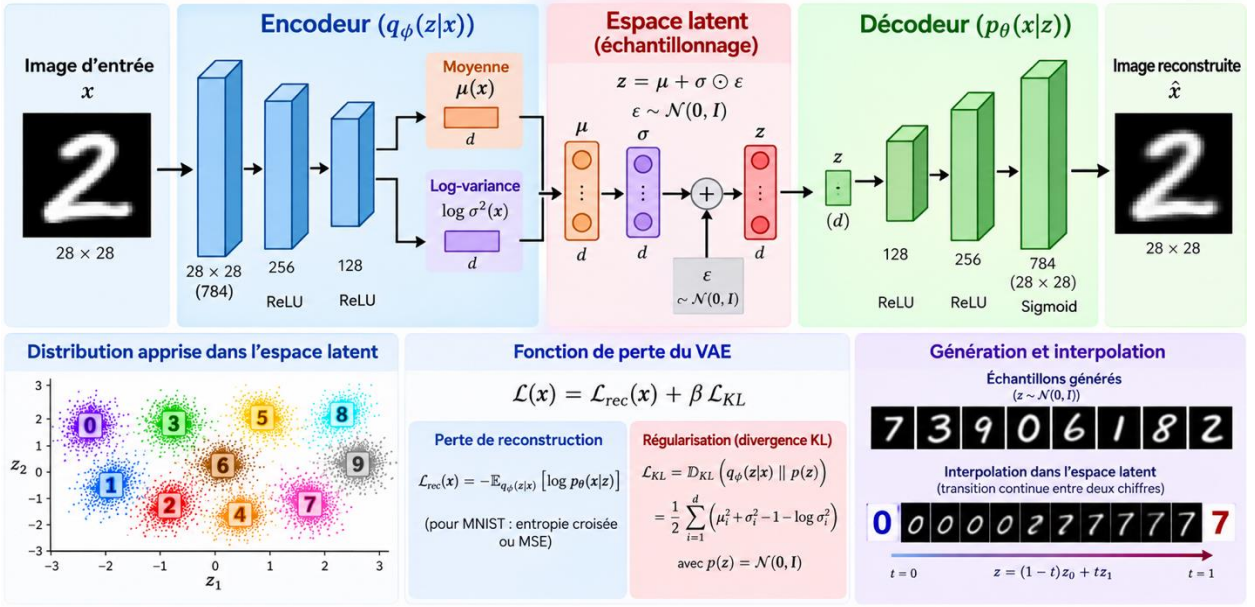
$$[\ p(z)=N(0,I)\ ]$$

- C'est une gaussienne centrée en zéro, avec une variance de 1 dans chaque dimension.
- Puis on pénalise l'écart entre :
  - la distribution produite par l'encodeur pour chaque image ;
  - cette distribution de référence.
- On mesure cet écart avec la **divergence de Kullback-Leibler**, ou **divergence KL**.
- La fonction de perte devient :

$$[\ \text{Perte VAE Perte de reconstruction} + \text{Divergence KL}\ ]$$

Les deux termes ont des rôles complémentaires :

Terme	Ce qu'il encourage
Reconstruction	Garder l'information spécifique à chaque image
Divergence KL	Rendre les distributions latentes compatibles avec une référence commune

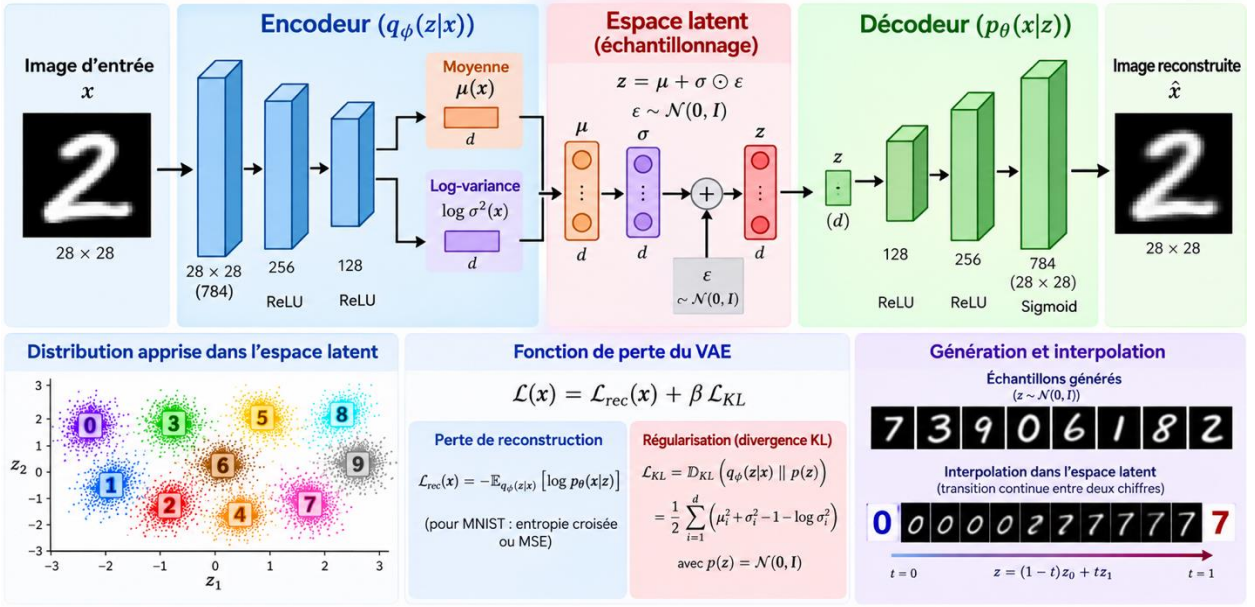


- La régularisation ne cherche pas à faire disparaître toute différence entre les images.
- Si tous les nuages devenaient exactement identiques, le code latent ne permettrait plus de distinguer un « 3 » d'un « 8 ».
- Le modèle doit donc trouver un compromis :
  - **Des codes assez différents pour reconstruire les images, mais assez organisés pour permettre de nouveaux tirages.**
- Une régularisation trop forte peut conduire le décodeur à ignorer le code latent : c'est un risque appelé **effondrement du posterior**.

Modèle Génératif : Variational Auto Encoder (VAE)

Pourquoi peut-on ensuite générer de nouvelles images ?

- Après l'entraînement, on n'a plus besoin de fournir une image à l'encodeur.
- On fait simplement :
  - Tirage de z dans N(0, I) → Décodeur → Nouvelle image
  - La régularisation a encouragé les codes utilisés pendant l'entraînement à occuper les régions où cette distribution de référence tire ses valeurs.
  - Le décodeur a donc davantage de chances de recevoir un code compatible avec ce qu'il a appris.
  - Cela **favorise** la génération d'images plausibles, sans garantir que chaque tirage produira un beau chiffre.



Modèle Génératif : Variational Auto Encoder (VAE)

```python
class VAE(nn.Module):
    def __init__(self, latent_dim=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Linear(256, 64),
            nn.ReLU()
        )
        self.fc_mu = nn.Linear(64, latent_dim)
        self.fc_logvar = nn.Linear(64, latent_dim)
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 256),
            nn.ReLU(),
            nn.Linear(256, 784),
            nn.Sigmoid()
        )
```

```python
def encode(self, x):
    h = self.encoder(x)
    mu = self.fc_mu(h)
    logvar = self.fc_logvar(h)
    return mu, logvar
```

```python
def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + std * eps
```

```python
def decode(self, z):
    x_hat = self.decoder(z)
    return x_hat.view(-1, 1, 28, 28)
```

```python
def forward(self, x):
    mu, logvar = self.encode(x)
    z = self.reparameterize(mu, logvar)
    x_hat = self.decode(z)
    return x_hat, mu, logvar
```

```python
vae = VAE(latent_dim=2).to(device)
vae
```

Reparameterization trick : Échantillonner $z = \mu + \sigma \odot \epsilon$ En conservant la rétropropagation

$$\epsilon \sim \mathcal{N}(0, I) \qquad z \sim \mathcal{N}(\mu, \sigma^2)$$

Modèle Génératif : Variational Auto Encoder (VAE)

Loss du VAE

```
def vae_loss(x_hat, x, mu, logvar, beta=1.0):
    rec_loss = F.binary_cross_entropy(
        x_hat,
        x,
        reduction="sum"
    )

    kl_loss = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - logvar.exp()
    )

    return rec_loss + beta * kl_loss, rec_loss, kl_loss
```

Le VAE combine deux termes :

Reconstruction $\mathcal{L}_{rec}$

Régularisation de l'espace latent

$$D_{KL} = -\frac{1}{2}\sum\left(1 + \log\sigma^2 - \mu^2 - \sigma^2\right)$$

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \beta D_{KL}$$

Modèle Génératif : Variational Auto Encoder (VAE)

```
vae_optimizer = torch.optim.Adam(vae.parameters(), lr=1e-3)
def train_vae(model, loader, epochs=20, beta=1.0):
    model.train()
    for epoch in range(epochs):
        total_loss = 0
        total_rec = 0
        total_kl = 0
        for x, _ in loader:
            x = x.to(device)
            x_hat, mu, logvar = model(x)
            loss, rec, kl = vae_loss(x_hat, x, mu,logvar, beta=beta)
            vae_optimizer.zero_grad()
            loss.backward()
            vae_optimizer.step()
            total_loss += loss.item()
            total_rec += rec.item()
            total_kl += kl.item()
        n = len(loader.dataset)
        print(f"Epoch {epoch+1}/{epochs} |Loss={total_loss/n:.2f}|Rec={total_rec/n:.2f} |KL={total_kl/n:.2f}")
train_vae(vae, train_loader, epochs=20)
```

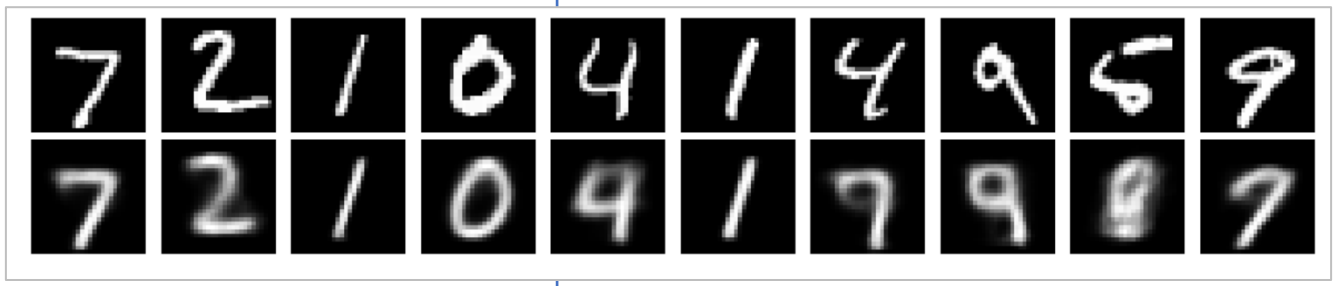
VAE Training

```
Epoch 1/20 | Loss=144.90 | Rec=138.61 | KL=6.29
Epoch 2/20 | Loss=144.70 | Rec=138.40 | KL=6.30
Epoch 3/20 | Loss=144.19 | Rec=137.86 | KL=6.33
Epoch 4/20 | Loss=143.91 | Rec=137.54 | KL=6.37
Epoch 5/20 | Loss=143.66 | Rec=137.27 | KL=6.39
Epoch 6/20 | Loss=143.36 | Rec=136.95 | KL=6.41
Epoch 7/20 | Loss=142.99 | Rec=136.56 | KL=6.43
Epoch 8/20 | Loss=143.13 | Rec=136.68 | KL=6.45
Epoch 9/20 | Loss=142.65 | Rec=136.17 | KL=6.48
Epoch 10/20 | Loss=142.58 | Rec=136.08 | KL=6.50
Epoch 11/20 | Loss=142.25 | Rec=135.74 | KL=6.51
Epoch 12/20 | Loss=141.74 | Rec=135.21 | KL=6.53
Epoch 13/20 | Loss=141.64 | Rec=135.07 | KL=6.56
Epoch 14/20 | Loss=141.60 | Rec=135.02 | KL=6.58
Epoch 15/20 | Loss=141.21 | Rec=134.62 | KL=6.59
Epoch 16/20 | Loss=141.68 | Rec=135.10 | KL=6.58
Epoch 17/20 | Loss=141.31 | Rec=134.71 | KL=6.60
Epoch 18/20 | Loss=141.13 | Rec=134.51 | KL=6.62
Epoch 19/20 | Loss=140.81 | Rec=134.18 | KL=6.62
Epoch 20/20 | Loss=140.79 | Rec=134.14 | KL=6.65
```

Modèle Génératif : Variational Auto Encoder (VAE)

Reconstruction avec le VAE

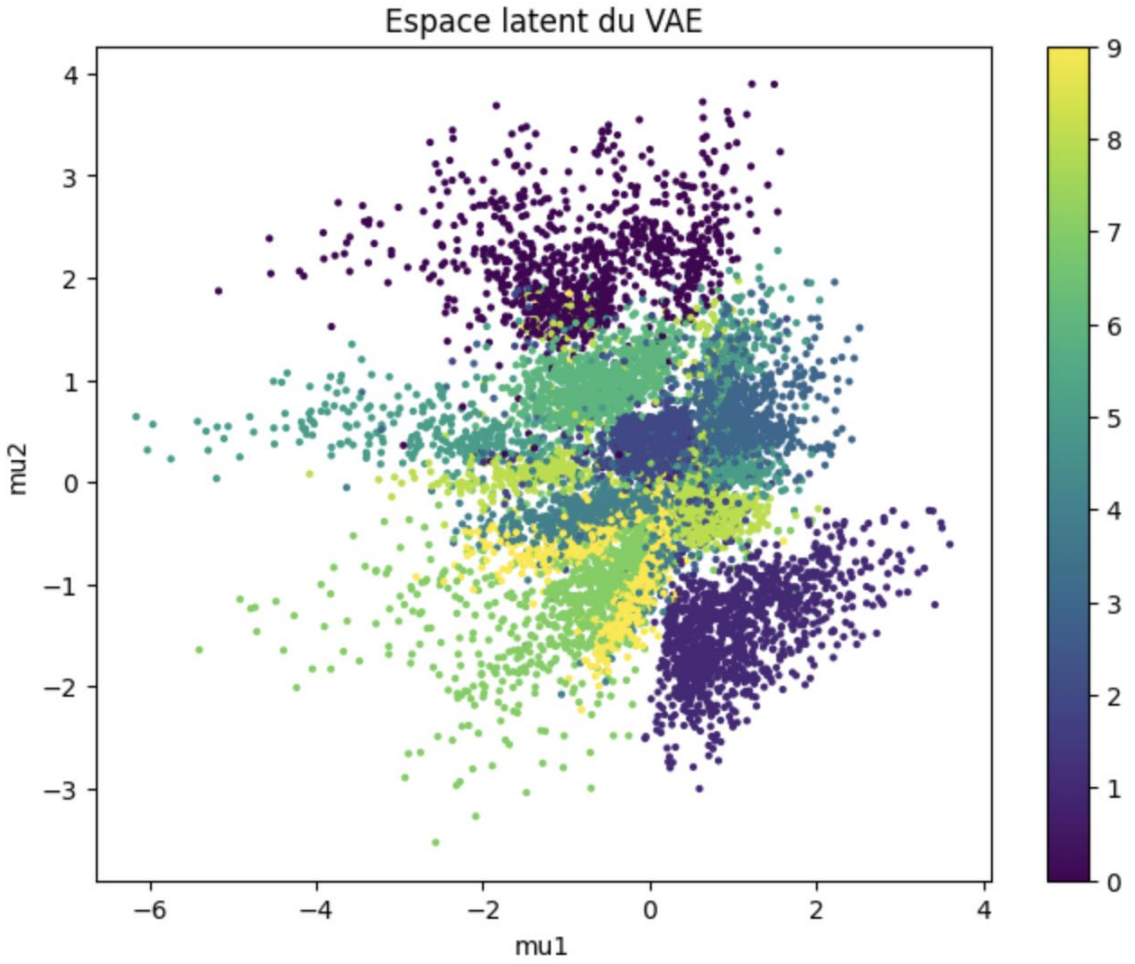
```
vae.eval()
x, _ = next(iter(test_loader))
x = x[:10].to(device)
with torch.no_grad():
    x_hat, _, _ = vae(x)
fig, axes = plt.subplots(2, 10, figsize=(15, 3))
for i in range(10):
    axes[0, i].imshow(x[i].cpu().squeeze(), cmap="gray")
    axes[0, i].axis("off")
    axes[1, i].imshow(x_hat[i].cpu().squeeze(), cmap="gray")
    axes[1, i].axis("off")
axes[0, 0].set_ylabel("Original")
axes[1, 0].set_ylabel("VAE")
plt.tight_layout()
plt.show()
```



Modèle Génératif : Variational Auto Encoder (VAE)

Visualisation de l'espace Latent

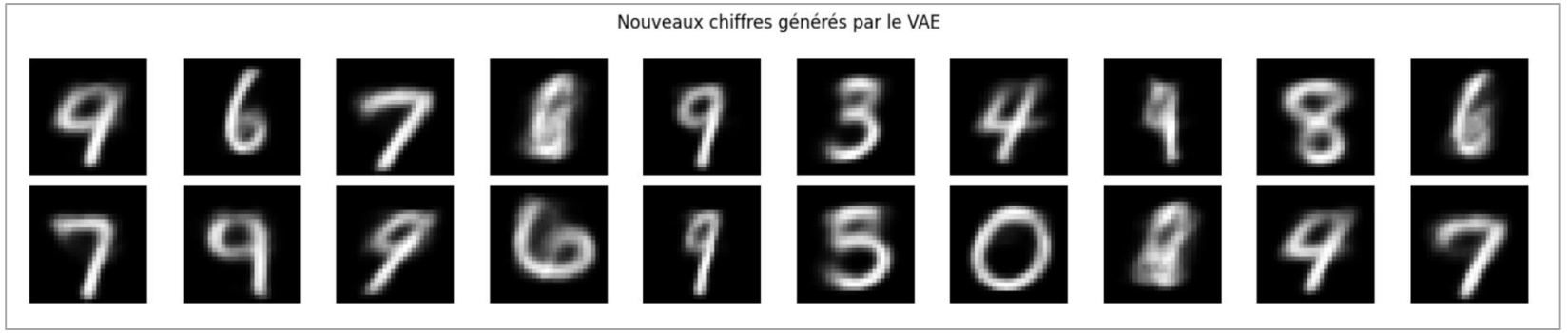
```
vae.eval()
mus = []
ys = []
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(device)
        mu, logvar = vae.encode(x)
        mus.append(mu.cpu())
        ys.append(y)
MU = torch.cat(mus).numpy()
Y = torch.cat(ys).numpy()
plt.figure(figsize=(8, 6))
scatter = plt.scatter( MU[:, 0], MU[:, 1], c=Y, s=4)
plt.xlabel("mu1")
plt.ylabel("mu2")
plt.title("Espace latent du VAE")
plt.colorbar(scatter)
plt.show()
```



Modèle Génératif : Variational Auto Encoder (VAE)

Générer de nouvelles images

```
vae.eval()
with torch.no_grad():
    z = torch.randn(20, 2).to(device)
    samples = vae.decode(z).cpu()
fig, axes = plt.subplots(2, 10, figsize=(15, 3))
for i, ax in enumerate(axes.flat):
    ax.imshow(samples[i].squeeze(), cmap="gray")
    ax.axis("off")
plt.suptitle("Nouveaux chiffres générés par le VAE")
plt.tight_layout()
plt.show()
```



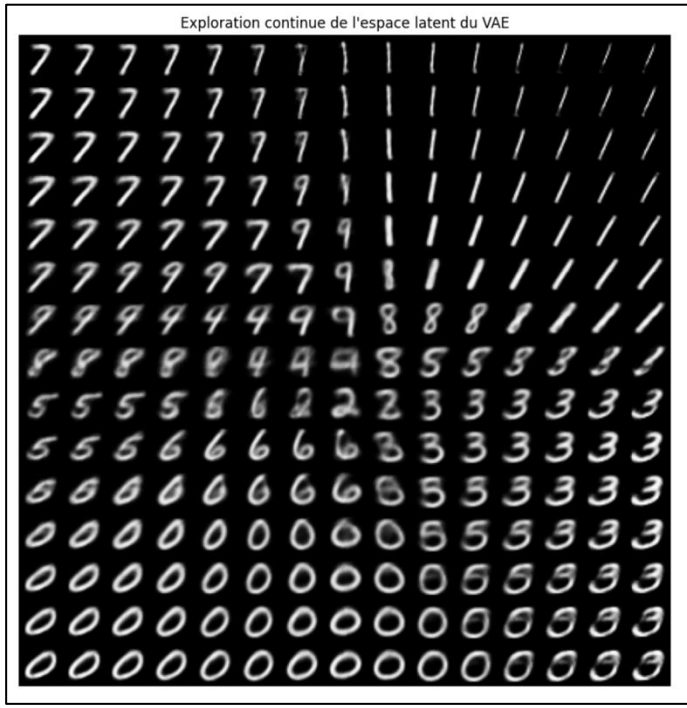
Modèle Génératif : Variational Auto Encoder (VAE)

Explorer manuellement l'espace latent

```
vae.eval()
grid_x = np.linspace(-3, 3, 15)
grid_y = np.linspace(-3, 3, 15)
figure = np.zeros((28 * 15, 28 * 15))
with torch.no_grad():
    for i, yi in enumerate(grid_y):
        for j, xi in enumerate(grid_x):
            z = torch.tensor(
                [[xi, yi]],
                dtype=torch.float32,
                device=device
            )
            x_hat = vae.decode(z)
            digit = x_hat.cpu().numpy()[0, 0]
            figure[
                i*28:(i+1)*28,
                j*28:(j+1)*28
            ] = digit
```

- Ce code permet de **visualiser ce qu'un VAE a appris à générer** en explorant son espace latent à deux dimensions.
- L'idée générale est simple :
  - On choisit plusieurs points dans l'espace latent. Pour chaque point, le décodeur génère une image. On rassemble ensuite toutes ces images dans une grande mosaïque.
  - Ici, on génère **15 × 15 = 225 images**, chacune de taille **28 × 28 pixels**.

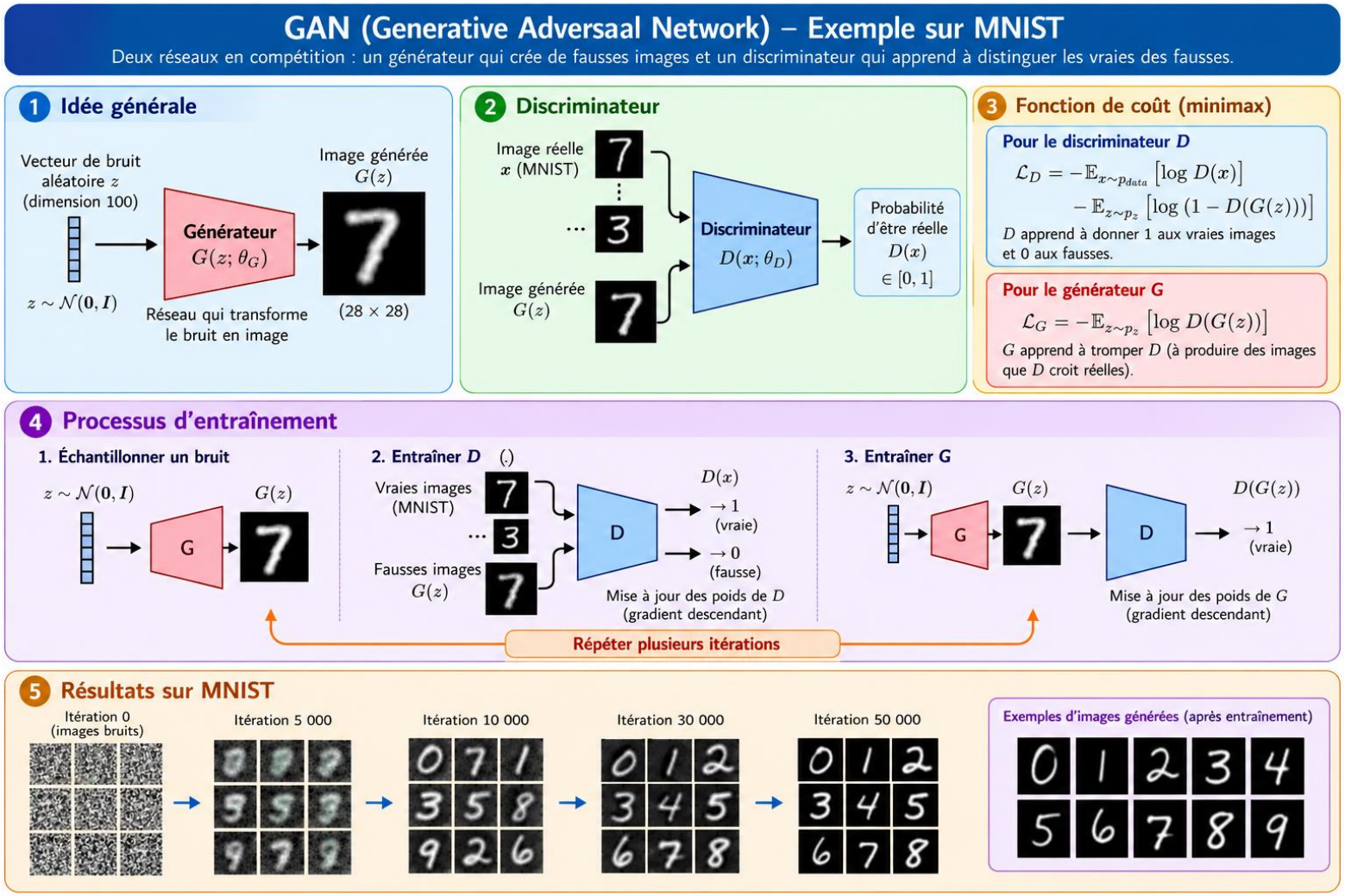
```
plt.figure(figsize=(10, 10))
plt.imshow(figure, cmap="gray")
plt.axis("off")
plt.title("Exploration continue de l'espace
latent du VAE")
plt.show()
```



Modèle Génératif : Generative Adversarial Network (GAN)

Explorer manuellement l'espace latent

- Un **GAN** (*Generative Adversarial Network*), ou **Réseau Antagoniste Génératif**, apprend à produire de nouvelles données ressemblant à celles d'un jeu d'entraînement.
- Il repose sur deux réseaux entraînés avec des objectifs opposés :
  - le **générateur**, qui crée de fausses données ;
  - le **discriminateur**, qui apprend à distinguer les données réelles des données générées.
- Le générateur joue le rôle d'un faussaire, tandis que le discriminateur joue celui d'un expert chargé de détecter les contrefaçons. Chacun pousse l'autre à progresser



Modèle Génératif : Generative Adversarial Network (GAN)

Explorer manuellement l'espace latent

```
from pathlib import Path
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

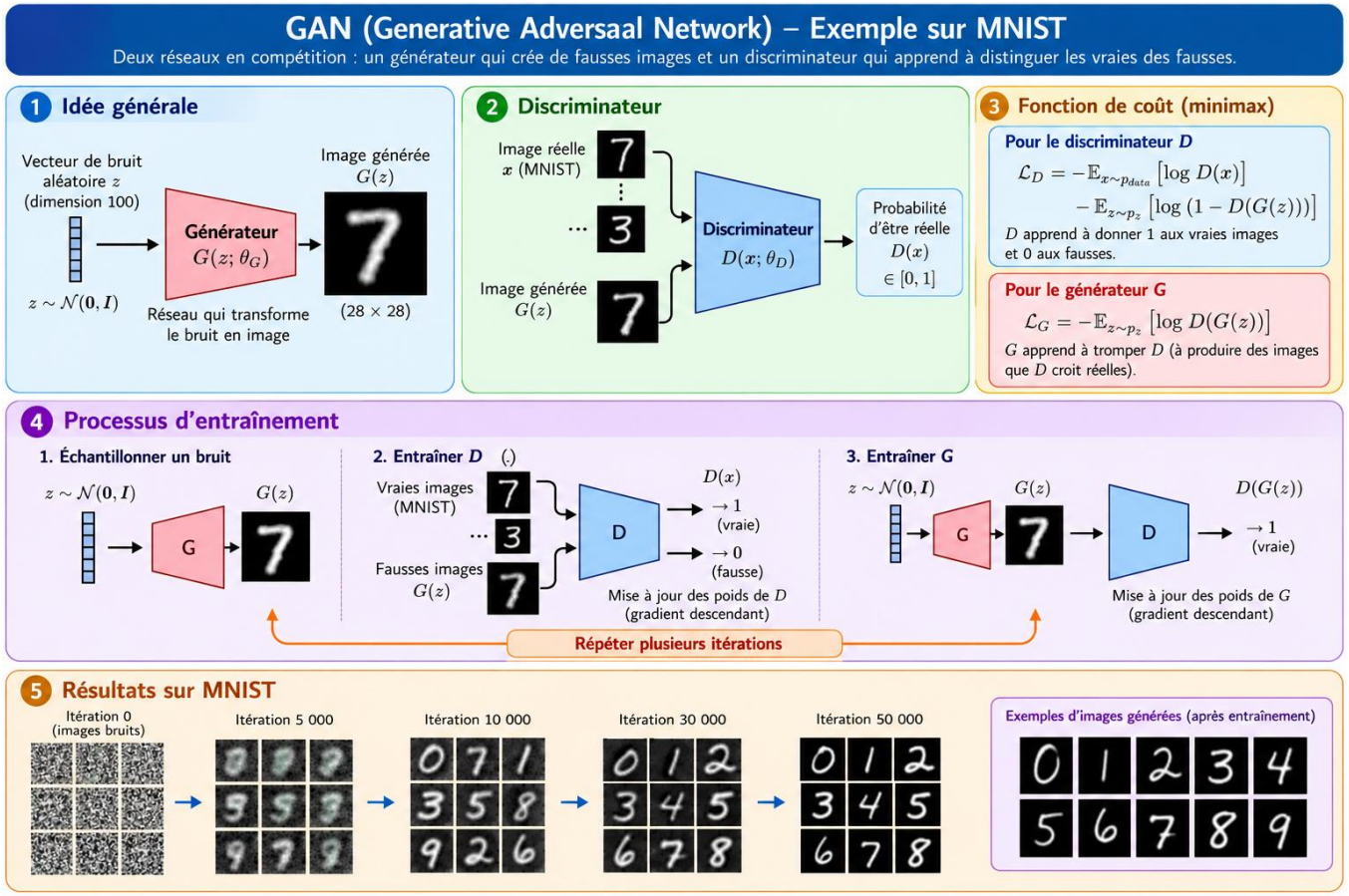
torch.manual_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

latent_dim = 100
image_dim = 28 * 28
batch_size = 128
epochs = 20
learning_rate = 0.0002

output_dir = Path("images_generees")
output_dir.mkdir(exist_ok=True)

print(f"Appareil utilisé : {device}")
```



Modèle Génératif : Generative Adversarial Network (GAN)

Préparation des données

```
from pathlib import Path
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from torchvision.utils import save_image

torch.manual_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

latent_dim = 100
image_dim = 28 * 28
batch_size = 128
epochs = 20
learning_rate = 0.0002

output_dir = Path("images_generees")
output_dir.mkdir(exist_ok=True)

print(f"Appareil utilisé : {device}")
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)),
])

dataset = datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=transform,
)

loader = DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=True,
)
```

Modèle Génératif : Generative Adversarial Network (GAN)

Generator et Discriminator

```
class Generator(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.LeakyReLU(0.2),

            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),

            nn.Linear(512, image_dim),
            nn.Tanh(),
        )

    def forward(self, noise):
        return self.network(noise)


generator = Generator(latent_dim).to(device)
```

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(image_dim, 512),
            nn.LeakyReLU(0.2),

            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),

            nn.Linear(256, 1),
        )

    def forward(self, images):
        return self.network(images)


discriminator = Discriminator().to(device)
```

```
criterion = nn.BCEWithLogitsLoss()
optimizer_g = torch.optim.Adam(
    generator.parameters(),
    lr=learning_rate,
    betas=(0.5, 0.999),
)
optimizer_d = torch.optim.Adam(
    discriminator.parameters(),
    lr=learning_rate,
    betas=(0.5, 0.999),
)
```

Modèle Génératif : Generative Adversarial Network (GAN)

Entrainer le Discriminator et le Generator

```
fixed_noise = torch.randn(64, latent_dim, device=device)
for epoch in range(epochs):
    generator.train()
    discriminator.train()
    total_loss_d, total_loss_g = 0.0, 0.0
    for real_images, _ in loader:
        # [B, 1, 28, 28] -> [B, 784]
        real_images = real_images.to(device).flatten(start_dim=1)
                current_batch_size = real_images.size(0)
        real_labels = torch.ones(
            current_batch_size, 1, device=device
        )
        fake_labels = torch.zeros(
            current_batch_size, 1, device=device
        )
        # A. Entraîner le discriminateur
        discriminator.requires_grad_(True)
        optimizer_d.zero_grad(set_to_none=True)
        # Évaluer les vraies images.
        real_scores = discriminator(real_images)
        loss_real = criterion(real_scores, real_labels)
        # Créer de fausses images à partir de bruit.
        noise = torch.randn(
            current_batch_size, latent_dim, device=device
        )
        fake_images = generator(noise)
        # Couper le gradient vers G pendant cette étape.
        fake_scores = discriminator(fake_images.detach())
        loss_fake = criterion(fake_scores, fake_labels)
        loss_d = (loss_real + loss_fake) / 2
        loss_d.backward()
        optimizer_d.step()
```

```
Époque 16/20 | Loss D : 0.3792 | Loss G : 1.9155
Époque 17/20 | Loss D : 0.4148 | Loss G : 1.7215
Époque 18/20 | Loss D : 0.4344 | Loss G : 1.6382
Époque 19/20 | Loss D : 0.4532 | Loss G : 1.5679
Époque 20/20 | Loss D : 0.4649 | Loss G : 1.5327
```

```
            # B. Entraîner le générateur
        discriminator.requires_grad_(False)
        optimizer_g.zero_grad(set_to_none=True)
        # Nouvelle évaluation avec D
        fake_scores = discriminator(fake_images)
        # G veut que ses images soient classées "réelles".
        loss_g = criterion(fake_scores, real_labels)
        loss_g.backward()
        optimizer_g.step()
        total_loss_d += loss_d.item()
        total_loss_g += loss_g.item()
    print(
        f"Époque {epoch + 1:02d}/{epochs} | "
        f"Loss D : {total_loss_d / len(loader):.4f} | "
        f"Loss G : {total_loss_g / len(loader):.4f}"
    )
     # C. Sauvegarder des exemples
enerator.eval()
ith torch.no_grad():
    samples = generator(fixed_noise)
    samples = samples.view(-1, 1, 28, 28)

    # [-1, 1] -> [0, 1] pour enregistrer les images.
    samples = (samples + 1) / 2

    save_image(
        samples,
        output_dir / f"epoch_{epoch + 1:03d}.png",
        nrow=8,
    )

# Sauvegarder les paramètres appris.
torch.save(generator.state_dict(), "generator_mnist.pt")
```

Modèle Génératif : Generative Adversarial Network (GAN)

Générer de Nouvelles images

```
# Générer des images
generator.eval()

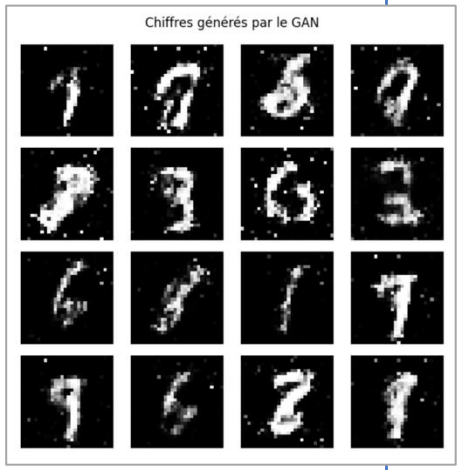
with torch.no_grad():
    new_noise = torch.randn(16, latent_dim, device=device)

    new_images = generator(new_noise)
    new_images = new_images.view(-1, 1, 28, 28)
    new_images = (new_images + 1) / 2

    save_image(
        new_images,
        output_dir / "nouveaux_chiffres.png",
        nrow=4,
    )
```

```
import matplotlib.pyplot as plt
import torch
generator.eval()
with torch.no_grad():
    noise = torch.randn(16, latent_dim, device=device)
    images = generator(noise)
    # [16, 784] -> [16, 1, 28, 28]
    images = images.reshape(-1, 1, 28, 28)
    # Convertir les pixels de [-1, 1] vers [0, 1]
    images = ((images + 1) / 2).clamp(0, 1)
    # Matplotlib utilise les données sur CPU
    images = images.cpu()
# Afficher une grille de 4 × 4
fig, axes = plt.subplots(4, 4, figsize=(6, 6))
for ax, image in zip(axes.flat, images):
    ax.imshow(image.squeeze(0).numpy(), cmap="gray", vmin=0, vmax=1)
    ax.axis("off")

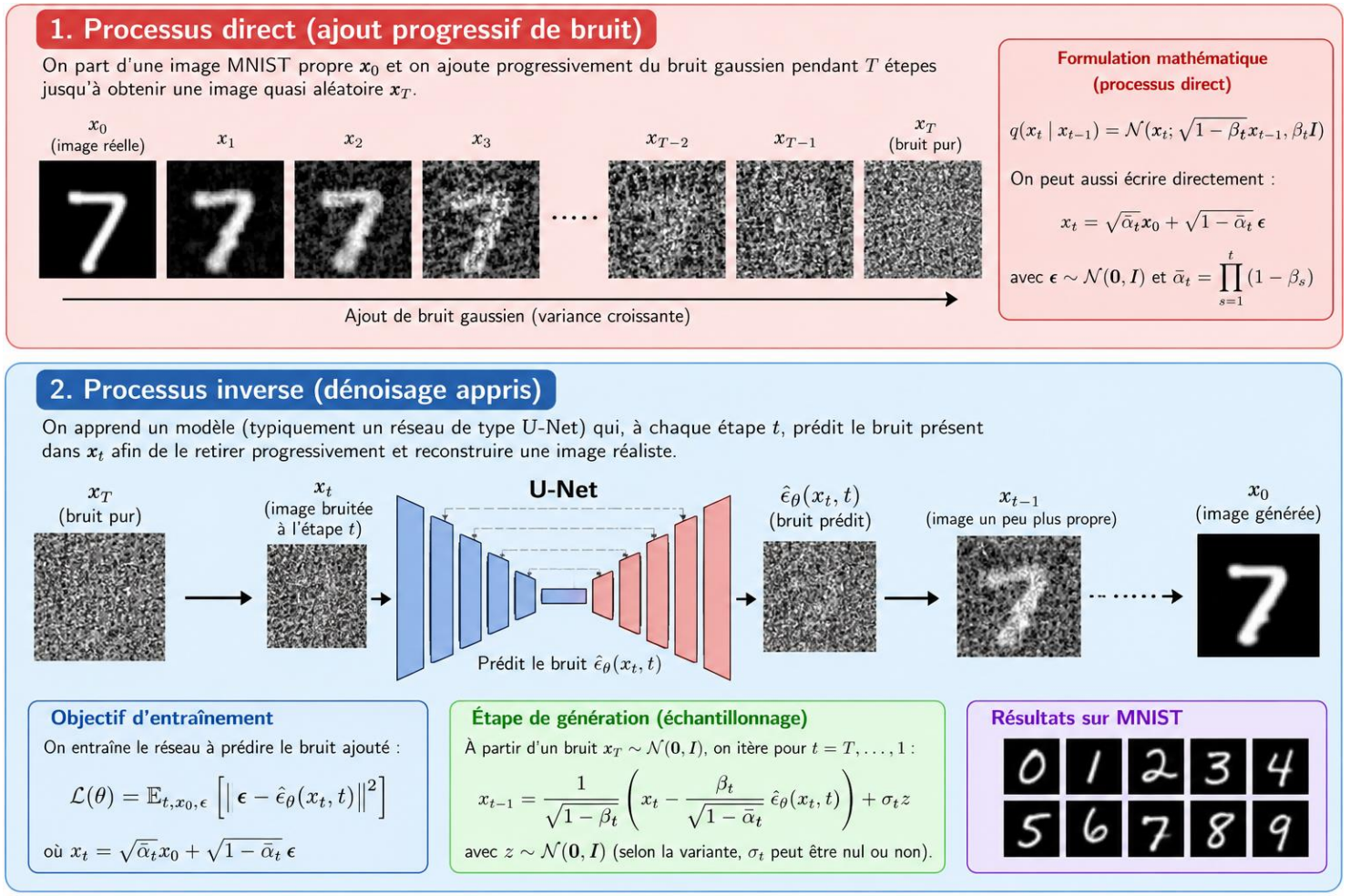
fig.suptitle("Chiffres générés par le GAN")
plt.tight_layout()
plt.show()
```



Modèle Génératif : Modèles de diffusion

Principe de base

- Un **modèle de diffusion** est un modèle génératif qui apprend à créer des données, par exemple des images, en **retirant progressivement du bruit**.
- **Idée centrale :** on apprend au modèle à débruiter des images, puis on utilise cette compétence pour transformer du bruit aléatoire en une nouvelle image.
- **Deux processus complémentaires**
  - **La diffusion directe : ajouter du bruit:** On part d'une image réelle et on lui ajoute progressivement du bruit gaussien. À mesure que le bruit augmente, les détails puis la structure de l'image disparaissent.
  - **La diffusion inverse : apprendre à débruiter:** Un réseau apprend à guider le chemin inverse : C'est ce processus inverse qui permet de générer de nouvelles images.



Modèle Génératif : Modèles de diffusion

Preparer les données

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
print("Périphérique :", device)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

```
train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)
train_loader = DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda")
)
```

Modèle Génératif : Modèles de diffusion

Ajout du bruit

```
T = 200
def cosine_beta_schedule(timesteps, s=0.008):
    steps = torch.linspace(0, timesteps, timesteps + 1)
    alpha_bar = torch.cos(
     ((steps / timesteps + s) / (1 + s)) * math.pi / 2
    ).pow(2)
    alpha_bar = alpha_bar / alpha_bar[0]
    betas = 1 - alpha_bar[1:] / alpha_bar[:-1]
    return betas.clamp(min=1e-4, max=0.999)
betas = cosine_beta_schedule(T).to(device)
alphas = 1.0 - betas
alpha_bars = torch.cumprod(alphas, dim=0)
alpha_bars_prev = torch.cat([
    torch.ones(1, device=device),
    alpha_bars[:-1]
])
```

```
posterior_variance = (
    betas * (1 - alpha_bars_prev) / (1 - alpha_bars)
)
def extract(values, t):
    """Sélectionner un coefficient par image """
    return values[t].view(-1, 1, 1, 1)
def add_noise(x0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x0)
    signal = extract(alpha_bars.sqrt(), t)
    noise_scale = extract((1 - alpha_bars).sqrt(), t)
    xt = signal * x0 + noise_scale * noise
    return xt, noise
```

Modèle Génératif : Modèles de diffusion

Observer la dégradation progressive d'une image

```
x0, _ = train_dataset[0]
x0 = x0.unsqueeze(0).to(device)
times = [0, 40, 80, 120, 160, 199]
noise = torch.randn_like(x0)
fig, axes = plt.subplots(1, len(times) + 1, figsize=(14, 2))
axes[0].imshow(
    x0[0, 0].cpu(), cmap="gray", vmin=-1, vmax=1
)
axes[0].set_title("Original")
axes[0].axis("off")
for ax, step in zip(axes[1:], times):
    t = torch.tensor([step], device=device)
    xt, _ = add_noise(x0, t, noise)
    ax.imshow( xt[0, 0].cpu(), cmap="gray", vmin=-1, vmax=1 )
    ax.set_title(f"t = {step}")
    ax.axis("off")
plt.tight_layout()
plt.show()
```

